hyperbolix: Hyperbolic Deep Learning in JAX

Timo Klein *timo.klein@univie.ac.at*
Faculty of Computer Science, University of Vienna, Vienna, Austria
UniVie Doctoral School Computer Science, University of Vienna, Vienna, Austria

Thomas Lang *thomas.lang@univie.ac.at*
Faculty of Computer Science, University of Vienna, Vienna, Austria
UniVie Doctoral School Computer Science, University of Vienna, Vienna, Austria

Yllka Velaj *yllka.velaj@univie.ac.at*
Faculty of Computer Science, University of Vienna, Vienna, Austria
ds:UniVie, University of Vienna, Vienna, Austria

Sebastian Tschiatschek *sebastian.tschiatschek@univie.ac.at*
Faculty of Computer Science, University of Vienna, Vienna, Austria
ds:UniVie, University of Vienna, Vienna, Austria

arXiv:2609.28248v2 [cs.LG] 14 Oct 2026

Abstract

We present **hyperbolix**, an open-source library for hyperbolic deep learning in JAX, built on Flax NNX. To our knowledge, it is the first comprehensive, general-purpose hyperbolic deep learning library in JAX. It includes six manifolds with a common interface: Euclidean space, the Poincaré ball, the hyperboloid, the κ-stereographic model, mixed-curvature product spaces, and the proper velocity space. We implement layer families that cover linear layers, convolutions, attention, normalization, positional encoding, regression, and vector quantization. These building blocks span methods ranging from Ganea's original hyperbolic neural networks to recent fully hyperbolic architectures such as Hypformer and Lorentzian ResNet. Additionally, hyperbolix contains Riemannian optimizers implemented as optax transformations, wrapped distributions, and hyperbolic dimensionality-reduction techniques. Its API uses idiomatic JAX: Manifolds are stateless, with curvature being passed at call time, while manifold operations act on single points, with `jax.vmap` enabling batch operations. The precision of every checked operation is tested against a closed-form NumPy/SciPy transcription from the source paper or a finite difference, for both float32 and float64. On the hyperboloid, standard formulas for two-point operations, such as the distance, lose precision far from the origin, because they subtract two large, nearly equal terms. hyperbolix replaces these subtractions with cancellation-free formulas that stay accurate in float32 at distances where prior implementations return NaN. hyperbolix is available under the MIT license at `https://github.com/timoklein/hyperbolix`.

1 Introduction

Deep learning reaches practitioners through frameworks such as PyTorch (Paszke et al., 2019) or the JAX ecosystem (Bradbury et al., 2018; Heek et al., 2026). Operations readily available in these frameworks, in turn, shape the methods researchers develop. Both PyTorch and JAX/Flax are Euclidean by construction: tensors live in $\mathbb{R}^n$, and common deep learning components such as linear layers, residual connections, and optimizer steps rely on Euclidean operations like affine transformations and vector addition. The assumption that data lives in a vector space is rarely made explicit because it rarely constrains what can be modeled. Hierarchical data violates that assumption. Examples include natural language, where words fall into semantic categories, and taxonomic classification, where each class is embedded within a broader superclass. Both impose a tree structure that a flat vector space cannot represent efficiently. The number of nodes in

a hierarchy grows exponentially with depth. Hyperbolic space, a Riemannian manifold of constant negative curvature, matches this growth rate: its volume expands exponentially with radius, making it the natural geometry for tree-like data (Ungar, 2009; Sarkar, 2011). But hyperbolic space is not a vector space, so operations like vector addition do not transfer directly. Hyperbolic deep learning has had to rebuild them: Ganea et al. (2018) introduced the first hyperbolic linear layer and softmax classifier, and the family has grown to include reformulated layers (Shimizu et al., 2021; Shi et al., 2026; van der Klis et al., 2026), convolutional backbones (Bdeir et al., 2024; van Spengler et al., 2023a; He et al., 2025c), transformers and self-attention (Chen et al., 2022; Yang et al., 2024; He et al., 2025a), vision–language models (Pal et al., 2025; Yoshikawa & Matsubara, 2025), as well as mixed-curvature product spaces (Gu et al., 2019; Skopek et al., 2020). A more thorough overview of current architectures and trends can be found in the numerous surveys discussing the literature, e.g., Mettes et al. (2024) or He et al. (2025b).

Which of these methods are readily available to practitioners depends on the framework they use. In PyTorch, geoopt (Kochurov et al., 2020) is the most comprehensive library, containing manifolds, neural network layers, and Riemannian optimizers. HypLL (van Spengler et al., 2023b) focuses on the Poincaré ball, providing hyperbolic layers. Geomstats (Miolane et al., 2020) covers general Riemannian geometry across backends but is not a deep learning library. JAX has Riemannian optimizers too: Rieoptax (Utpala et al., 2022), RiemannAX (mary, 2024), and GeoJAX (You, 2026) all provide them, but none offer a general-purpose hyperbolic layer library. Rieoptax contains a handful of undocumented Poincaré and Lorentz layers but has been inactive since 2023, while GeoJAX and RiemannAX ship no hyperbolic layers at all. Thus, the only option for JAX users in need of hyperbolic linear layers, attention blocks, or normalization layers is to implement them from scratch or to hand-port an existing implementation. However, such a port is a tricky endeavor due to common numerical stability issues.

To close this gap in the JAX ecosystem, we present hyperbolix: a hyperbolic deep learning library for JAX (Bradbury et al., 2018) and Flax NNX (Heek et al., 2026). It implements six manifolds with a unified interface: three common models of hyperbolic geometry, the Poincaré ball, the hyperboloid, and the proper velocity model, alongside a signed-curvature κ-stereographic model (Bachmann et al., 2020), Euclidean space, and mixed-curvature manifolds (Gu et al., 2019). Exact isometries connect the three hyperbolic models, so a representation learned in one transfers to another without retraining. Our `nn_layers` package provides 76 implementations that cover the linear, convolution, attention, normalization, positional encoding, regression, and vector quantization layers of standard architectures. Around the layers, we also implement Riemannian Adam and SGD (Bonnabel, 2013; Bécigneul & Ganea, 2019) written as optax transformations (DeepMind et al., 2020), wrapped normal distributions (Nagano et al., 2019), as well as HoroPCA (Chami et al., 2021) and CO-SNE (Guo et al., 2022) as dimensionality reduction techniques. hyperbolix is available under the MIT license at `https://github.com/timoklein/hyperbolix`.

Key Contributions

1. **The library.** To our knowledge, hyperbolix is the first comprehensive, general-purpose hyperbolic deep learning library for JAX (Section 3, Table 2). Its API adapts manifold operations to JAX's functional paradigm, illustrated by a worked example that converts a Euclidean model to its hyperbolic counterpart (Section 2, Figure 1).
2. **Systematic correctness verification.** Every checked operation is compared against an independently derived reference rather than another library call (Section 5).
3. **Higher numerical accuracy than existing libraries.** On the Poincaré ball, hyperbolix matches or exceeds the round-trip precision of geoopt and HypLL. On the hyperboloid, we replace every Minkowski inner-product subtraction with a cancellation-free formula (Bläsius et al., 2016; Celinska-Kopczynska & Kopczynski, 2024), staying at float32 rounding level up to geodesic radius 10, where geoopt already returns NaN (Section 4, Figures 2 and 3).

2 Design and API

This section presents the API of hyperbolix, starting with the design principles that make it compatible with JAX transformations. Next, we introduce learnable curvature, Riemannian optimization, and initialization.

2.1 Core design principles

Every manifold operation in hyperbolix is a pure function, meaning its output depends only on its inputs, and it modifies no external state. This makes hyperbolix composable with `jax.jit`, `jax.vmap`, and `jax.grad` (Bradbury et al., 2018), implemented through the following three design decisions.

Manifold operations take unbatched inputs. A distance takes two arrays of shape `(dim,)` and returns a scalar; $\exp_0$ takes a tangent vector and returns a manifold point, again with no batch axis. Batching is an explicit `jax.vmap` written by the caller, who has to state mapped axes. A batched implementation would fix the layout of an operation, requiring callers to reshape inputs for pairwise, broadcast, and per-example uses of the same distance. The unbatched design leaves the choice of layout to the caller.

Manifolds are plain Python classes, not `nnx.Modules`. Because curvature is passed into every call, a manifold instance holds no NNX state, and Flax NNX does not track it as part of the module graph. One instance can therefore be shared across all layers, including those within `nnx.scan` or `nnx.fori_loop`, which reject objects whose state is referenced from multiple places.

Curvature is passed at call time. We write c for the curvature parameter of a hyperbolic manifold, which has constant sectional curvature $-c$ for $c > 0$. The κ-stereographic manifold takes a signed κ whose sign selects hyperbolic, Euclidean, or spherical geometry (Bachmann et al., 2020). Because c is an argument rather than a construction-time state, it can be a Python float (a compile-time constant under `jit`) or a traced `jax.Array`, and the same manifold instance serves both fixed and learned curvature.

Figure 1 shows a Euclidean Flax NNX model beside its hyperboloid counterpart using hyperbolix. Three key differences are visible. First, the Euclidean features are lifted onto the manifold in `__call__`. Prepending a zero time coordinate maps them to the tangent space at the origin, and $\exp_0$ sends the result to the hyperboloid under an explicit `jax.vmap`. Second, each `HTCLinear` layer (Yang et al., 2024) applies a Euclidean linear transformation to the input point and then recomputes the time coordinate, ensuring that the output lies on the hyperboloid. The layer optionally accepts separate input and output curvatures, enabling a model to change curvature between layers. Third, the classifier is a hyperbolic multinomial logistic regression head that uses points on the hyperboloid as inputs. The model class, parameter handling, and training loop are unchanged from the Euclidean version, so these three changes are the full cost of the conversion.

2.2 Learnable curvature

Curvature is often better treated as a parameter than a hyperparameter. `LearnableCurvature` is a standalone `nnx.Module` held by the model that yields c on call. The manifold itself stays stateless. `LearnableCurvature` stores a single real-valued scalar corresponding to the curvature through one of three parameterizations. The default, `log`, sets $c = \exp(r)$ for an unconstrained parameter r. Each step in r then changes c by a constant factor rather than by a constant amount, which suits curvatures spanning orders of magnitude (Yoshikawa & Matsubara, 2025). `softplus` sets $c = \text{softplus}(r)$, guaranteeing $c > 0$ with derivative $\partial c/\partial r = \sigma(r) \in (0, 1)$ (Kochurov et al., 2020). `identity` is the signed parameterization required for reaching the Euclidean and spherical values of the κ-stereographic manifold. The curvature is additionally optionally clamped to a fixed interval. Because r is an unconstrained real number, a plain `optax.adam` (DeepMind et al., 2020) can update it.

2.3 Optimization

hyperbolix ships `riemannian_adam` and `riemannian_sgd` as `optax.GradientTransformations`, so they compose with Optax and can plug into `nnx.Optimizer`. Manifold-valued parameters are detected automatically: `ManifoldParam`, an `nnx.Param` subclass, carries the manifold and curvature, and `mark_manifold_param`

```python
import jax
from flax import nnx


class MLP(nnx.Module):
    def __init__(self, rngs: nnx.Rngs):
        self.embed = nnx.Linear(784, 32, rngs=rngs
            )
        self.fc1 = nnx.Linear(32, 64, rngs=rngs)
        self.fc2 = nnx.Linear(64, 64, rngs=rngs)
        self.head = nnx.Linear(64, 10, rngs=rngs)

    def __call__(self, x):
        x = jax.nn.relu(self.embed(x))
        x = jax.nn.relu(self.fc1(x))
        x = jax.nn.relu(self.fc2(x))
        return self.head(x)
```

(a) Flax NNX (Euclidean)

```python
import jax, jax.numpy as jnp
from flax import nnx
from hyperbolix.manifolds import Hyperboloid
from hyperbolix.nn_layers import (HTCLinear,
    HypRegressionHyperboloid, hrc_relu)

H = Hyperboloid()  # plain class, not a Module

class HypMLP(nnx.Module):
    def __init__(self, rngs: nnx.Rngs):
        self.embed = nnx.Linear(784, 32, rngs=rngs
            )
        self.fc1 = HTCLinear(33, 64, rngs=rngs)
        self.fc2 = HTCLinear(65, 64, rngs=rngs)
        self.head = HypRegressionHyperboloid(
            H, 65, 10, rngs=rngs, input_space="
                manifold")

    def __call__(self, x, c=1.0):
        x = jax.nn.relu(self.embed(x))
        x = jax.vmap(lambda v: H.expmap_0(
            jnp.concatenate([jnp.zeros(1), v]), c)
                )(x)
        x = hrc_relu(self.fc1(x, c_in=c, c_out=c),
              c, c)
        x = hrc_relu(self.fc2(x, c_in=c, c_out=c),
              c, c)
        return self.head(x, c)
```

(b) hyperbolix (hyperboloid)

Figure 1: **Converting a Euclidean MNIST MLP to a hybrid hyperbolic network.** Both models train with the same loop and a plain `optax.adam`, because both parameterize their weights in Euclidean space (Section 2.3). Three differences are visible in (b): the Euclidean features are projected onto the manifold via $\exp_0$ under an explicit `jax.vmap`, the curvature c is passed as a call-time argument to every hyperbolic layer, and a hyperbolic regression head replaces the linear output layer. The dimensions 33 and 65 are ambient hyperboloid dimensions: 32 and 64 spatial coordinates, respectively, plus one time coordinate, which is fully determined by the space component.

wraps an existing parameter. Every unmarked parameter receives the standard Euclidean update in the same transformation.

Most models avoid Riemannian SGD because modern layers parametrize manifold parameters as Euclidean parameters mapped onto the manifold (Shimizu et al., 2021; Shi et al., 2026). Their gradients are therefore Euclidean, making `optax.adam` the correct optimizer. Riemannian optimization is for parameters that genuinely live on the manifold: embedding tables optimized directly on the ball (Nickel & Kiela, 2017; Chen et al., 2025a) and legacy Ganea-style Poincaré layers whose bias is parameterized on the ball while the weight matrix stays Euclidean (Ganea et al., 2018).

2.4 Initialization

Generic He or Xavier initialization fails in hyperbolic layers in two ways. If the initialization weights are too large, the layer's outputs land near the boundary of the Poincaré ball or far up the hyperboloid, where distances and their gradients become `NaN` within a few steps. If the weights are too small, the per-layer gain below one compounds as $\mathcal{O}\left(\text{gain}^{\text{depth}}\right)$ until the outputs of distinct inputs become numerically indistinguishable in the working dtype, collapsing the network to a constant function. Appendix E lists the default initialization of each layer family and its rationale, and Section 4 covers the underlying precision limits.

3 The hyperbolix Library

hyperbolix is built around six manifolds that share a common interface. The neural network layers, Riemannian optimizers, distributions, and dimensionality-reduction methods build on them. This section describes

Table 1: Contents of the `nn_layers` package. A ✓ marks manifold coverage. The full inventory with source attributions is in Appendix A.

Slot	Variants	Hyperboloid	Poincaré	Proper Velocity
Linear	11	✓	✓	✓
Convolution	7	✓	✓	✓
Attention	4	✓		
Normalization & dropout	12	✓	✓	✓
Residual, pooling, primitives	15	✓	✓	
Positional encoding	3	✓		
Regression / MLR	7	✓	✓	✓
Vector quantization	3		✓	
Activations	13	✓	✓	
Interface[†]	1	✓	✓	
Total	76			

[†] Hybrid regularizer (`HyperPPFeatureScaling` (Klein et al., 2025); not itself on-manifold).

the manifolds first, then the layers, and the remaining packages. Finally, we compare hyperbolix to existing libraries.

3.1 Manifolds

All manifolds implement a common interface of 19 operations (Appendix D), so a user-defined manifold with the same methods works with the Riemannian optimizers and other manifold-generic components without subclassing any library class. The three hyperbolic models, the Poincaré ball, the hyperboloid, and the proper velocity space, pose different trade-offs. The hyperboloid yields larger gradient magnitudes than the Poincaré ball, aiding optimization (Mishne et al., 2023), but its standard two-point formulas suffer catastrophic cancellation in the Minkowski inner product (Section 4). The Poincaré ball is conformal to Euclidean space and supports 2D visualization, but its conformal factor diverges at the boundary resulting in unavoidable numerical issues. The proper velocity (PV) model represents points as unconstrained vectors in $\mathbb{R}^n$, avoiding $\exp_0$ and $\log_0$ projection steps (Chen et al., 2026b). In hyperbolix, two-point operations (distance, parallel transport, gyrovector addition) leverage the fact that the PV $\leftrightarrow$ hyperboloid isometry is exact, allowing us to use the hyperboloid's cancellation-free arithmetic for PV operations (Section 4). Exact isometries connect all three pairs of hyperbolic models (Poincaré $\leftrightarrow$ hyperboloid, proper velocity $\leftrightarrow$ Poincaré, proper velocity $\leftrightarrow$ hyperboloid), so a representation learned in one model transfers to another without retraining. The stereographic manifold (Bachmann et al., 2020) unifies hyperbolic, Euclidean, and spherical geometry under one signed curvature. Bachmann et al. (2020) parameterize this curvature as κ, with $\kappa < 0$ hyperbolic, $\kappa = 0$ Euclidean, and $\kappa > 0$ spherical. We instead expose $c \doteq -\kappa$, so $c > 0$ is hyperbolic, matching our Poincaré ball convention. Our manifold operations for the κ-stereographic model are analytically smooth at $c = 0$ because the implementation evaluates their limits at $c = 0$ via Taylor expansions. Therefore, the gradient with respect to c remains finite at the Euclidean point. With the `identity` parameterization of Section 2, learnable curvature can cross zero during training rather than being pinned to a fixed sign. `ProductManifold` (Gu et al., 2019) builds mixed-curvature product spaces from any of the above, takes curvature information as a per-factor tuple, and implements the same generic interface.

3.2 Neural network layers

The `nn_layers` package provides 44 layer classes, 31 functions, and one data type. Appendix A lists the paper behind each. It contains linear, convolution, attention, normalization, positional-encoding, regression, activation, and vector quantization, comprising standard operations in deep learning. For a particular manifold and network layer, there can be multiple implementations with differing properties. This is for instance

the case for the Poincaré linear layer: Ganea et al. (2018) parameterize it with a Euclidean weight matrix and a manifold bias, which requires Riemannian optimization. The HNN++ reformulation (Shimizu et al., 2021) moves all parameters off the manifold, so a standard `optax.adam` suffices. Similar parameterization trade-offs recur throughout the library: the convolution, attention, and normalization families each offer variants that target different manifolds and pose distinct trade-offs among fidelity, numerical stability, and speed. In appendices A and B we present the full list of layers and methods and where they were initially introduced.

3.3 Beyond layers

The `optim` package provides `riemannian_adam` and `riemannian_sgd` as ordinary optax (DeepMind et al., 2020) `GradientTransformation`s that detect manifold parameters automatically. Section 2.3 covers the implementation.

We further implement three distributions to support generative and probabilistic models: wrapped normals (Nagano et al., 2019) on the Poincaré ball and the hyperboloid, and a uniform distribution on the ball.

For dimensionality reduction and visualization, `decomposition` implements HoroPCA (Chami et al., 2021) and CO-SNE (Guo et al., 2022).

`utils` exposes Gromov δ-hyperbolicity estimation (Khrulkov et al., 2020), which measures how tree-like a dataset is, together with the domain-clamped math primitives that Section 4 examines.

The documentation helps users choose among these implementations: for each layer family, such as linear or convolutional layers, a table states when to pick each layer. A separate numerical stability guide explains the failure modes of Section 4 and how to avoid them, for example, when to switch to float64.

3.4 Comparison to existing libraries

To our knowledge, hyperbolix is the first comprehensive, general-purpose *hyperbolic deep learning* library in JAX. While JAX already has Riemannian optimization (Rieoptax (Utpala et al., 2022), RiemannAX (mary, 2024), GeoJAX (You, 2026)), none of these libraries offers a general-purpose hyperbolic layer library. In PyTorch, geoopt (Kochurov et al., 2020) and HypLL (van Spengler et al., 2023b) are the closest counterparts; Geomstats (Miolane et al., 2020) covers general Riemannian geometry, but does not focus on deep learning. Table 2 compares hyperbolix against these libraries feature by feature. Appendix C surveys the JAX libraries in detail.

4 Numerical Stability

Distances in hyperbolic space grow exponentially with radius, so a fixed floating-point budget resolves exponentially less of the space with distance from the origin. On the Poincaré ball with curvature $c > 0$, the Riemannian metric is conformal to the Euclidean metric: for a tangent vector v, the conformal factor $\lambda(x) = 2/(1 - c\|x\|^2)$ rescales its Euclidean norm to its Riemannian norm at x, $\|v\|_x = \lambda(x)\|v\|_2$. It diverges as $\|x\|$ approaches the boundary radius $1/\sqrt{c}$. With float32 precision, spacing near 1.0 is about 1.19×10^{-7}, so a boundary-adjacent quantity like $1 - c\|x\|^2$ loses most of its significant digits before any hyperbolic operation is executed.

The hyperboloid has its own distinct failure mode, caused by the Minkowski inner product $\langle x, y\rangle_L = -x_0 y_0 + \langle x_s, y_s\rangle$, where x_0 is a point's time coordinate and x_s its spatial part. Almost every hyperboloid operation evaluates this inner product, including distance, tangent norm, parallel transport, and gyrovector addition. Its minus sign subtracts the product $x_0 y_0$ from the spatial inner product $\langle x_s, y_s\rangle$. For two nearby points at geodesic radius a, the hyperboloid constraint forces $\|x_s\| \approx x_0$, so $x_0 y_0$ and $\langle x_s, y_s\rangle$ both grow as e^{2a} (Mishne et al., 2023; van der Klis et al., 2026). These two large terms are nearly equal for nearby points. Because floating-point numbers store each term to a fixed number of digits, subtracting them erases the digits they share. The remaining absolute error is proportional to e^{2a}: for a large radius a, it can exceed the distance the

Table 2: Feature comparison of hyperbolix and other existing manifold and hyperbolic deep learning libraries. P: Poincaré ball, H: hyperboloid, PV: proper velocity. Layer counts are the classes exported from each library's public neural network namespace. geoopt's released version exposes none. Geomstats is included as a general geometric-statistics library. JAX libraries with Riemannian-geometry support (Rieoptax, RiemannAX, GeoJAX) are surveyed in Appendix C. †manify's 0.2.1 wheel on PyPI is incomplete and cannot be imported, we use git commit `18028ec`.

Feature	**hyperbolix**	**geoopt**	**HypLL**	**manify**	**Geomstats**
Version	1.3.0	0.5.1	0.1.1	0.2.1†	2.8.0
Framework	JAX/ Flax NNX	PyTorch	PyTorch	PyTorch (on geoopt)	NumPy, Autograd, PyTorch
Hyp. models	3 (P, H, PV)	2 (P, H)	1 (P)	2 (P, H)	3 (P, H, half-sp.)
Signed curvature	✓	✓	×	✓	×
Product manifolds	✓	✓	×	✓	✓
Learnable curvature	✓	✓	✓	✓	×
Isometries	✓	✓(P↔H)	×	✓(P↔H)	✓
NN layers	✓(44)	×	✓(11)	✓(8)	×
Optimizers	Adam, SGD	Adam, SGD, LS, sparse variants	Adam, SGD	Adan; Adam, SGD via geoopt	Internal solvers
Wrapped dist.	P, H	P (sampling)	×	H (P: sampling only)	×
Dim. reduction	HoroPCA, CO-SNE	×	×	×	Tangent PCA, PGA
License	MIT	Apache-2.0	MIT	MIT	MIT

computation is meant to measure. We call this *ambient cancellation*. In float32, it exhausts the mantissa at $a \approx 9$ (float64: $a \approx 19$), even though the stored coordinates remain accurate to $a \approx 16$. Without mitigation, the error follows the predicted cancellation curve (Figure 2). geoopt returns NaN for distances at radii $a \approx 9$. An implementation that clamps the `acosh` input to its domain $[1, \infty)$ would return finite but wrong values after $a \approx 9$, instead.

hyperbolix replaces those clamps with one rule: never form the canceling difference. Every Minkowski quadratic form is rewritten as a sum of non-negative terms. For the distance between two hyperboloid points x and y with spatial norms $r_x = \|x_s\|$, $r_y = \|y_s\|$, unit directions $\hat{x} = x_s/r_x$, $\hat{y} = y_s/r_y$, and radii $a_x = \operatorname{arsinh}(\sqrt{c}\, r_x)$, $a_y = \operatorname{arcosh}(\sqrt{c}\, y_0)$, this gives

$$\sinh^2\!\left(\tfrac{\sqrt{c}\, d}{2}\right) \;=\; \sinh^2\!\left(\tfrac{a_x - a_y}{2}\right) \;+\; \tfrac{c}{4}\, r_x\, r_y\, \|\hat{x} - \hat{y}\|^2 \,, \tag{1}$$

where d is the geodesic distance. This yields the hyperbolic analog of the haversine identity, where the first term is the radial gap and the second the angular gap (Bläsius et al., 2016; Celinska-Kopczynska & Kopczynski, 2024). Both are non-negative, and neither subtracts large, nearly equal quantities. The same decomposition of tangent vectors into radial and perpendicular components applies to the tangent metric, parallel transport, and gyrovector addition. In float32, the remaining error comes from storing the points, not from the formula: **hyperbolix's median distance stays within a factor of** 8 **of the input-rounding error, the error that exact arithmetic on the float32-rounded points would still incur** (Figure 2). Beyond geodesic radius $a \approx 10$, the float32 distance error grows gradually rather than jumping to NaN: the median reaches 10^{-1} at $a = 16$ (Appendix H.7). On the Poincaré ball, domain guards on `atanh` and `tanh` keep the gradients finite at the boundary; the cancellation problem does not arise there because the ball's arithmetic does not pass through a Minkowski inner product. Figure 3 compares the $\log_0(\exp_0(v))$ round-trip error across tangent-vector norms r for geoopt (Kochurov et al., 2020), HypLL (van Spengler et al., 2023b), and hyperbolix. On the Poincaré ball, hyperbolix and HypLL are comparable in accuracy, while geoopt exhibits elevated error near the origin. On the hyperboloid, hyperbolix stays at rounding error level for all tangent-vector norms r from 10^{-3} to 20. In contrast, geoopt's error is five orders of magnitude larger (median 2.3×10^{-2} against 1.1×10^{-7} at $r = 10^{-3}$) near the origin. It matches hyperbolix only for $r \gtrsim 1$. HypLL does not implement the hyperboloid.

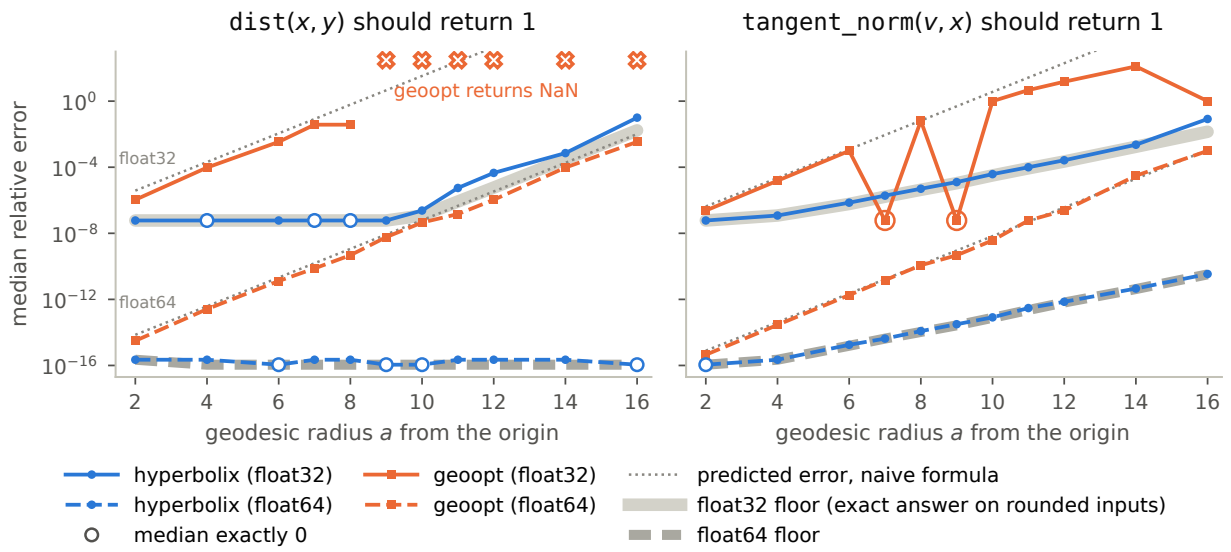

Figure 2: Median relative error of hyperboloid operations at curvature $c = 1$ in dimension 8, as a function of the geodesic radius a from the origin. We measure the distance between two points on one ray at unit separation, and the norm of a unit tangent vector at 45° to the ray. For both float32 and float64, hyperbolix tracks the input-rounding error. On the float32 distance past $a \approx 10$, it stays within 8× of it. For the tangent norm, geoopt's median is exactly 0 at $a = 7$ and 9 only because cancellation leaves it so few possible float32 results that more than half land exactly on 1. The rest are far off.

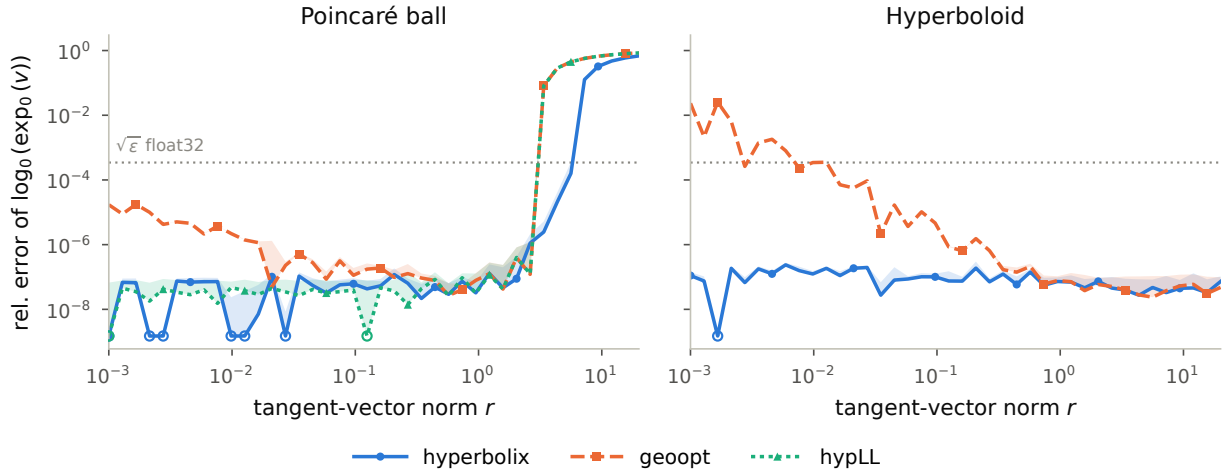

Figure 3: Expmap-logmap round trip stability of different libraries as a function of the tangent vector norm r. For the Poincaré ball, hyperbolix and HypLL are more stable than geoopt around the origin. On the hyperboloid, hyperbolix stays at the rounding level across the measured range. geoopt shows elevated error near the origin. HypLL does not implement hyperboloid operations.

Two conventions make stability checkable. First, tolerances are resolved through one function, `default_atol`(dtype) $= \sqrt{\texttt{finfo}(\text{dtype}).\texttt{eps}}$, which gives 3.45×10^{-4} in float32 and 1.49×10^{-8} in float64. Every manifold-membership and tangent-space check uses it unless the caller overrides. On the Poincaré ball, membership tests the dimensionless residual $c\|x\|^2 - 1$, whereas on the hyperboloid it compares the stored time coordinate against the value derived from the spatial part. Both residuals are dimensionless, so

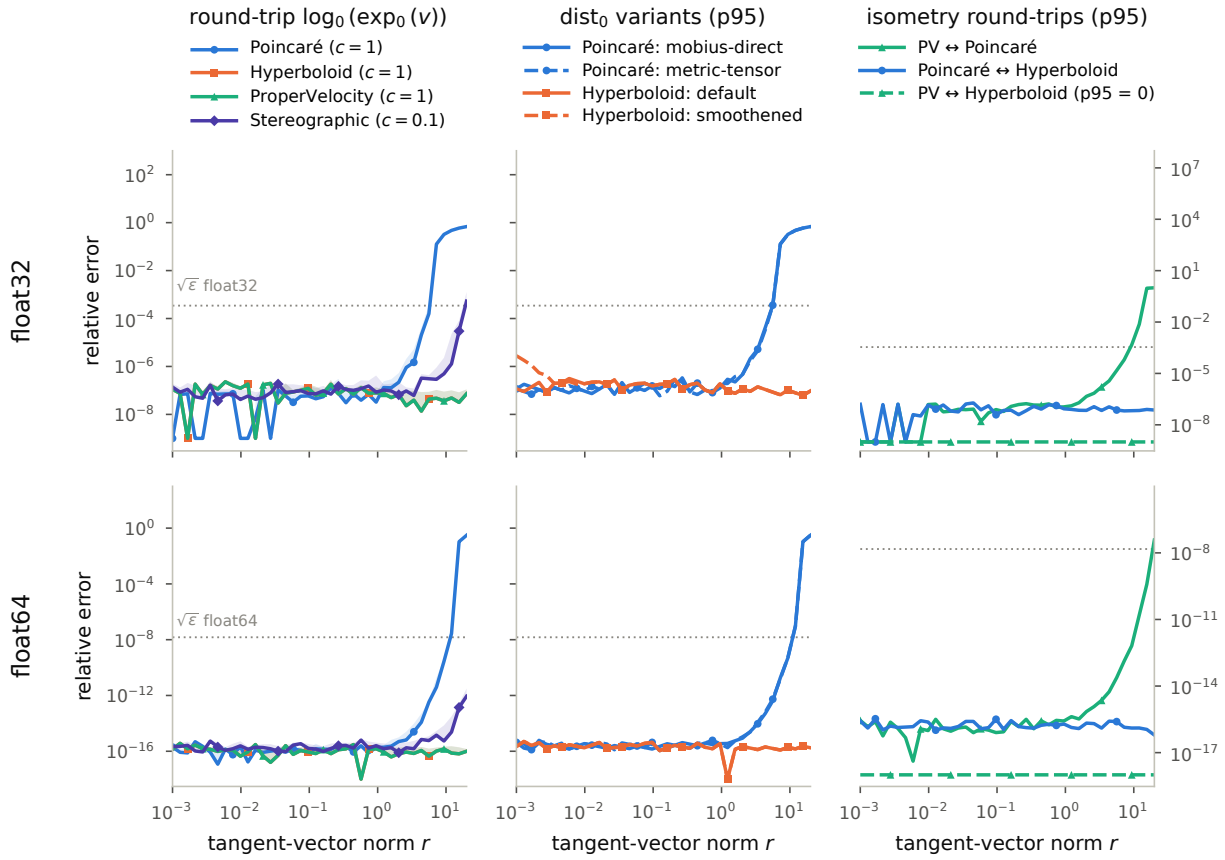

Figure 4: Relative error of hyperbolix operations in float32 (top) and float64 (bottom) as a function of the tangent-vector norm r, at curvature $c = 1$ in dimension 8. *Left:* origin round trip $\log_0(\exp_0(v))$. *Center:* dist_0 variants. *Right:* isometry round-trip between manifold models. The x axis is the tangent-vector norm. Geodesic distance is $2r$ on the ball and κ-stereographic model, r elsewhere. The PV $\to$ hyperboloid $\to$ PV round trip is exactly zero by construction.

the same tolerance applies uniformly across curvatures. Second, functions that admit several analytically equivalent formulations (e.g. distance computations) are selected by their `version_idx` argument, static under `jax.jit`. These different versions exist for the Poincaré ball and the hyperboloid, where they trade speed against stability or gradient properties.

For operations anchored at the origin, the Poincaré ball loses accuracy before the other models. At $c = 1$, its median round-trip error exceeds $\sqrt{\varepsilon}$ between geodesic distances 11 and 14.5 in float32, and 19 and 24 in float64. In contrast, the hyperboloid and proper-velocity models remain at rounding error level in both dtypes to the end of our measured range, at a geodesic distance of 20. In float32, the Poincaré ball's limit is set by hyperbolix's projection margin, which keeps points at least $\varepsilon^{0.75}$ inside the boundary and caps the ball at geodesic distance 12.6. These radii scale as $1/\sqrt{c}$. Two-point hyperboloid operations lose accuracy at smaller radii than origin operations (Figure 2). Users who need more accuracy than float32 provides can run the manifold operations in float64 while keeping parameters stored in float32. The manifold's `dtype` sets the compute precision and each layer's `param_dtype` the storage precision. `ProperVelocity` coordinates are unconstrained $\mathbb{R}^n$ (Chen et al., 2026b) and have no boundary to drift toward. In hyperbolix, their two-point operations pass through the exact isometry to the hyperboloid, inheriting its cancellation-free arithmetic. The numbers behind both figures are in Appendix H.4.

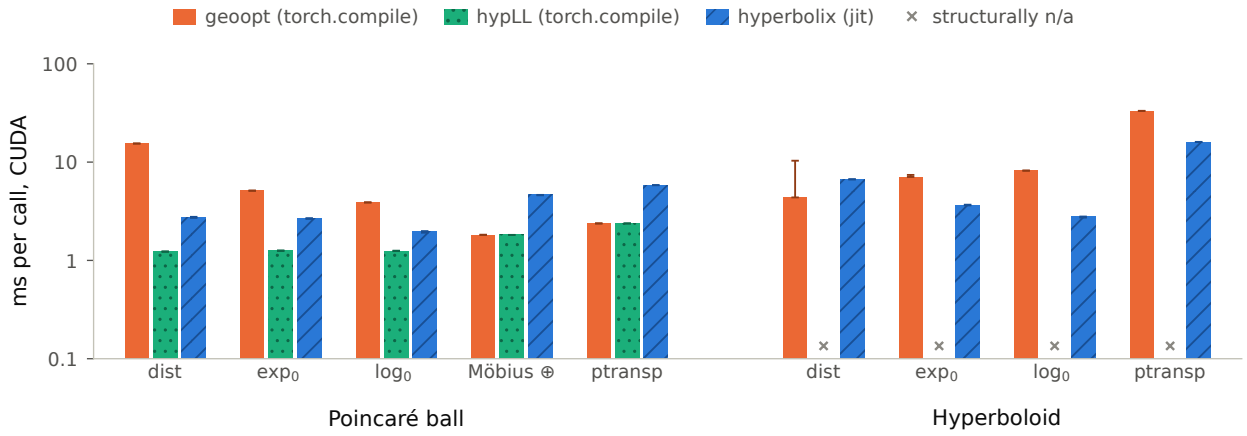

Figure 5: Wall-clock time per call for matched primitives on CUDA (H100, float32, batch 10^7, five launches at seed 0). geoopt (Kochurov et al., 2020) and HypLL (van Spengler et al., 2023b) use `torch.compile`, while hyperbolix uses `jax.jit` with `vmap`. Brackets span the five launches and bound machine and compilation state, not sampling variation. HypLL implements only the Poincaré ball. The y-axis is logarithmic with a 0.1 ms floor.

5 Correctness: Testing Against Independent Oracles

At curvature $c = 1$, a hyperbolic operation whose closed form introduces a bug to its $\sqrt{c}$ scaling term is identical to the correct operation because every $\sqrt{c}$ factor equals 1. At any other curvature, the error is silent: the operation still returns a point on the manifold, is still differentiable, and still trains. Shape checks, finiteness checks, and on-manifold checks all pass. Only a comparison against a known value catches the error, and such a comparison is worth exactly as much as the value it compares against. Every checked operation in hyperbolix is therefore tested against an *independent oracle*: a hand-derived closed form, a NumPy/SciPy transcription of the defining expression from the source paper, or a finite-difference estimate. No correctness assertion compares one hyperbolix call against another or against a second library (Kochurov et al., 2020; van Spengler et al., 2023b) because a self-consistent implementation passes its own cross-checks. Consequently, two libraries derived from the same reference code would inherit the same mistakes. Comparisons between hyperbolix implementations serve only as *consistency checks*: `jit` against eager execution, tangent-space against manifold input, and expmap-logmap round-trips. Our test suite comprises 1,038 test functions across 55 files, parametrized over float32 and float64. Appendix G details the tolerance conventions, the shared layer contract, and these consistency checks. A transcription oracle, however, verifies correctness versus the source paper, not the paper's correctness.

An illustrative example highlighting the benefits of this design is `Hyperboloid.log_radius_concat`, which uses the digamma function ψ to rescale the spatial part of each of N blocks of dimension d so that the expected log spatial radius is preserved. Swapping the two digamma arguments turns the intended shrinkage $\exp(\frac{1}{2}(\psi(d/2)-\psi(Nd/2))) \approx 1/\sqrt{N}$ into a concatenated radius that is $\mathcal{O}(N)$ too large, where N is the number of blocks and d the block dimension. The concatenation would inflate the radius that the rescaling exists to preserve. Generic on-manifold tests cannot catch this bug: the output is still a valid manifold point. The same swap is present in the "Intrinsic Lorentz Neural Network" paper and reference implementation (Shi et al., 2026). A test based on that implementation would have reproduced the error, whereas the oracle caught it due to it being derived from the identity $\mathbb{E}[\log \|v\|] = \frac{1}{2}(\psi(k/2) + \log 2)$ for $\|v\|^2 \sim \chi^2_k$, where χ^2_k is the Chi-squared distribution. Table 20 reports per-operation deviations between each hyperbolix call and its oracle over the standard sampling grids.

6 Performance

In contrast to Euclidean layers, hyperbolic layers incur additional pointwise overhead due to manifold operations. A Möbius linear map or a Lorentz distance evaluates transcendentals elementwise, then threads the result through a chain of pointwise steps (norms, clamps, conformal rescalings, re-projections) whose only purpose is to return a valid manifold point. Each of these steps is memory-bound: it moves more data than the arithmetic it performs would require. Under eager execution, each step also runs as its own kernel, so every step pays a separate kernel launch and a separate round trip to memory. `jax.jit` fuses many of these steps into fewer kernels, keeps intermediates in registers where possible, and removes the Python dispatch that otherwise dominates at small batch sizes (Bradbury et al., 2018). Because hyperbolix defines each manifold operation on a single point (Section 2), `jax.vmap` covers every batch shape from that one definition without hand-written broadcasting.

Figure 5 shows wall-clock time per call for matched primitives (distance, $\exp_0$, $\log_0$, Möbius addition, parallel transport) in float32 using CUDA on a batch of size 10^7. We compare geoopt (Kochurov et al., 2020), HypLL (van Spengler et al., 2023b), and hyperbolix. The large batch size is required to amortize dispatch overhead. geoopt and HypLL use `torch.compile` to match `jax.jit`, and hyperbolix operations are batched with `vmap`. The results show that HypLL is the fastest for the only manifold it implements, the Poincaré ball. geoopt and hyperbolix are broadly comparable in speed, with the winner dependent on the exact function being tested.

7 Limitations

The measurements of Sections 5–6 have a narrow scope. The oracle table (Table 20), the cross-library accuracy table (Table 21), the stability figures (Figures 3 and 4), and the speed comparison (Figure 5; Tables 22–25) are single-seed sweeps: every speed launch uses seed 0, so the brackets bound launch-to-launch machine and compilation state, and not sampling variation. We report no training-quality benchmarks: nothing here shows hyperbolix reaching a published accuracy on a published task.

Downstream evidence is thin. The library was developed and hardened within one research project; the two vector-quantization bottlenecks were ported from it. Wider use is what would expose interfaces that only appear general.

Two limits are structural. The float32 range ceiling of Section 4 follows from the exponential growth of the hyperbolic metric. For the hyperboloid, the cancellation-free formulas of Section 4 push the failure point outward without removing it. For the Poincaré ball at $c = 1$, past a geodesic radius of about 14 float64 remains the only remedy, and past about 24 it fails in turn. Coverage is also uneven across manifolds: the three attention layers and both positional encodings exist only on the hyperboloid, and both vector-quantization bottlenecks only on the Poincaré ball (Appendix A).

8 Conclusion

We present hyperbolix, a hyperbolic deep learning library for JAX and Flax NNX. To our knowledge, it is the first general-purpose hyperbolic deep learning library in JAX. hyperbolix comprises six manifolds using a common interface, layer families covering linear layers, convolutions, attention, normalization, positional encoding, regression, and vector quantization, Riemannian optimizers written as optax transformations. Its API is written to enable `jit`, `vmap`, and `scan`. The correctness of hyperbolix is based on the independent-oracle test suite of Section 5. The tests compare every checked operation against a hand-derived closed form, a transcription of the source paper, or a finite difference rather than against another library call. *hyperbolix's cancellation-free hyperboloid arithmetic keeps the float32 distance error within* $8\times$ *of the input-rounding error (Section 4).* We report comprehensive measurements, and make hyperbolix available under the MIT license at `https://github.com/timoklein/hyperbolix`. Contributions are welcome.

References

Gregor Bachmann, Gary Bécigneul, and Octavian Ganea. Constant curvature graph convolutional networks. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, volume 119 of *Proceedings of Machine Learning Research*, pp. 486–496. PMLR, 2020. URL http://proceedings.mlr.press/v119/bachmann20a.html.

Ahmad Bdeir, Kristian Schwethelm, and Niels Landwehr. Fully hyperbolic convolutional neural networks for computer vision. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL https://openreview.net/forum?id=ekz1hN5QNh.

Gary Bécigneul and Octavian-Eugen Ganea. Riemannian adaptive optimization methods. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019. URL https://openreview.net/forum?id=r1eiqi09K7.

Thomas Bläsius, Tobias Friedrich, Anton Krohmer, and Sören Laue. Efficient embedding of scale-free graphs in the hyperbolic plane. In Piotr Sankowski and Christos D. Zaroliagis (eds.), *24th Annual European Symposium on Algorithms, ESA 2016, Aarhus, Denmark, August 22-24, 2016*, volume 57 of *LIPIcs*, pp. 16:1–16:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2016. doi: 10.4230/LIPICS.ESA.2016.16. URL https://doi.org/10.4230/LIPIcs.ESA.2016.16.

Silvère Bonnabel. Stochastic gradient descent on riemannian manifolds. *IEEE Trans. Autom. Control.*, 58(9): 2217–2229, 2013. doi: 10.1109/TAC.2013.2254619. URL https://doi.org/10.1109/TAC.2013.2254619.

James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Yash Katariya, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL http://github.com/jax-ml/jax.

Tianci Bu, Chuanrui Wang, Hao Ma, Haoren Zheng, Xin Lu, and Tailin Wu. Ggball: Graph generative model on poincaré ball. *CoRR*, abs/2506.07198, 2025. doi: 10.48550/ARXIV.2506.07198. URL https://doi.org/10.48550/arXiv.2506.07198.

Dorota Celinska-Kopczynska and Eryk Kopczynski. Numerical aspects of hyperbolic geometry. In Leonardo Franco, Clélia de Mulatier, Maciej Paszynski, Valeria V. Krzhizhanovskaya, Jack J. Dongarra, and Peter M. A. Sloot (eds.), *Computational Science - ICCS 2024 - 24th International Conference, Malaga, Spain, July 2-4, 2024, Proceedings, Part VI*, volume 14837 of *Lecture Notes in Computer Science*, pp. 115–130. Springer, 2024. doi: 10.1007/978-3-031-63778-0_9. URL https://doi.org/10.1007/978-3-031-63778-0_9.

Ines Chami, Albert Gu, Dat Nguyen, and Christopher Ré. Horopca: Hyperbolic dimensionality reduction via horospherical projections. In Marina Meila and Tong Zhang (eds.), *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, volume 139 of *Proceedings of Machine Learning Research*, pp. 1419–1429. PMLR, 2021. URL http://proceedings.mlr.press/v139/chami21a.html.

Shangyu Chen, Pengfei Fang, Mehrtash Harandi, Trung Le, Jianfei Cai, and Dinh Phung. Hvq-vae: Variational auto-encoder with hyperbolic vector quantization. *Comput. Vis. Image Underst.*, 258(C), July 2025a. ISSN 1077-3142. doi: 10.1016/j.cviu.2025.104392. URL https://doi.org/10.1016/j.cviu.2025.104392.

Weize Chen, Xu Han, Yankai Lin, Hexu Zhao, Zhiyuan Liu, Peng Li, Maosong Sun, and Jie Zhou. Fully hyperbolic neural networks. In Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (eds.), *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022*, pp. 5672–5686. Association for Computational Linguistics, 2022. doi: 10.18653/V1/2022.ACL-LONG.389. URL https://doi.org/10.18653/v1/2022.acl-long.389.

Ziheng Chen, Yue Song, Xiaojun Wu, and Nicu Sebe. Gyrogroup batch normalization. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025b. URL https://openreview.net/forum?id=d1NWq4PjJW.

Ziheng Chen, Bernhard Schölkopf, and Nicu Sebe. Hyperbolic busemann neural networks. *CoRR*, abs/2602.18858, 2026a. doi: 10.48550/ARXIV.2602.18858. URL https://doi.org/10.48550/arXiv.2602.18858.

Ziheng Chen, Zihan Su, Bernhard Schölkopf, and Nicu Sebe. Proper velocity neural networks. In C. Vondrick, B. Hariharan, C. Raffel, L. Pinto, D. Yang, and A. Faust (eds.), *International Conference on Learning Representations*, volume 2026, pp. 122721–122760, 2026b. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/c78dd867a7501bbc39a1751ac80130b6-Paper-Conference.pdf.

DeepMind, Igor Babuschkin, Kate Baumli, Alison Bell, Surya Bhupatiraju, Jake Bruce, Peter Buchlovsky, David Budden, Trevor Cai, Aidan Clark, Ivo Danihelka, Antoine Dedieu, Claudio Fantacci, Jonathan Godwin, Chris Jones, Ross Hemsley, Tom Hennigan, Matteo Hessel, Shaobo Hou, Steven Kapturowski, Thomas Keck, Iurii Kemaev, Michael King, Markus Kunesch, Lena Martens, Hamza Merzic, Vladimir Mikulik, Tamara Norman, George Papamakarios, John Quan, Roman Ring, Francisco Ruiz, Alvaro Sanchez, Laurent Sartran, Rosalia Schneider, Eren Sezener, Stephen Spencer, Srivatsan Srinivasan, Miloš Stanojević, Wojciech Stokowiec, Luyu Wang, Guangyao Zhou, and Fabio Viola. The DeepMind JAX Ecosystem, 2020. URL http://github.com/google-deepmind.

Octavian-Eugen Ganea, Gary Bécigneul, and Thomas Hofmann. Hyperbolic neural networks. In Samy Bengio, Hanna M. Wallach, Hugo Larochelle, Kristen Grauman, Nicolò Cesa-Bianchi, and Roman Garnett (eds.), *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*, pp. 5350–5360, 2018. URL https://proceedings.neurips.cc/paper/2018/hash/dbab2adc8f9d078009ee3fa810bea142-Abstract.html.

Nabarun Goswami, Yusuke Mukuta, and Tatsuya Harada. Hypervq: Mlr-based vector quantization in hyperbolic space. *Trans. Mach. Learn. Res.*, 2025, 2025. URL https://openreview.net/forum?id=WgJgIULL9Q.

Albert Gu, Frederic Sala, Beliz Gunel, and Christopher Ré. Learning mixed-curvature representations in product spaces. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019. URL https://openreview.net/forum?id=HJxeWnCcF7.

Yunhui Guo, Haoran Guo, and Stella X. Yu. CO-SNE: dimensionality reduction and visualization for hyperbolic data. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022*, pp. 11–20. IEEE, 2022. doi: 10.1109/CVPR52688.2022.00011. URL https://doi.org/10.1109/CVPR52688.2022.00011.

Neil He, Rishabh Anand, Hiren Madhu, Ali Maatouk, Smita Krishnaswamy, Leandros Tassiulas, Menglin Yang, and Rex Ying. HELM: hyperbolic large language models via mixture-of-curvature experts. In Danielle Belgrave, Cheng Zhang, Laura N. Montoya, Hsuan-Tien Lin, Razvan Pascanu, Piotr Koniusz, Marzyeh Ghassemi, Nancy Chen, Iván Vladimir Meza Ruíz, and Arturo Loaiza-Bonilla (eds.), *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2025, NeurIPS 2025, San Diego, CA, USA, December 2-7, 2025 / Mexico City, Mexico, November 30 - December 5, 2025*, 2025a. URL http://papers.nips.cc/paper_files/paper/2025/hash/d1e2f808a51842eedaf6ef0099d716c6-Abstract-Conference.html.

Neil He, Hiren Madhu, Ngoc Bui, Menglin Yang, and Rex Ying. Hyperbolic deep learning for foundation models: A survey. In Luiza Antonie, Jian Pei, Xiaohui Yu, Flavio Chierichetti, Hady W. Lauw, Yizhou Sun, and Srinivasan Parthasarathy (eds.), *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining, V.2, KDD 2025, Toronto ON, Canada, August 3-7, 2025*, pp. 6021–6031. ACM, 2025b. doi: 10.1145/3711896.3736564. URL https://doi.org/10.1145/3711896.3736564.

Neil He, Menglin Yang, and Rex Ying. Lorentzian residual neural networks. In Yizhou Sun, Flavio Chierichetti, Hady W. Lauw, Claudia Perlich, Wee Hyong Tok, and Andrew Tomkins (eds.), *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining, V.1, KDD 2025, Toronto, ON, Canada, August 3-7, 2025*, pp. 436–447. ACM, 2025c. doi: 10.1145/3690624.3709292. URL https://doi.org/10.1145/3690624.3709292.

Jonathan Heek, Anselm Levskaya, Avital Oliver, Marvin Ritter, Bertrand Rondepierre, Andreas Steiner, and Marc van Zee. Flax: A neural network library and ecosystem for JAX, 2026. URL http://github.com/google/flax.

Valentin Khrulkov, Leyla Mirvakhabova, Evgeniya Ustinova, Ivan V. Oseledets, and Victor S. Lempitsky. Hyperbolic image embeddings. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2020, Seattle, WA, USA, June 13-19, 2020*, pp. 6417–6427. Computer Vision Foundation / IEEE, 2020. doi: 10.1109/CVPR42600.2020.00645. URL https://openaccess.thecvf.com/content_CVPR_2020/html/Khrulkov_Hyperbolic_Image_Embeddings_CVPR_2020_paper.html.

Timo Klein, Thomas Lang, Andrii Shkabrii, Alexander Sturm, Kevin Sidak, Lukas Miklautz, Claudia Plant, Yllka Velaj, and Sebastian Tschiatschek. Understanding and improving hyperbolic deep reinforcement learning. *CoRR*, abs/2512.14202, 2025. doi: 10.48550/ARXIV.2512.14202. URL https://doi.org/10.48550/arXiv.2512.14202.

Max Kochurov, Rasul Karimov, and Serge Kozlukov. Geoopt: Riemannian optimization in pytorch. *CoRR*, abs/2005.02819, 2020. URL https://arxiv.org/abs/2005.02819.

Aaron Lou, Isay Katsman, Qingxuan Jiang, Serge J. Belongie, Ser-Nam Lim, and Christopher De Sa. Differentiating through the fréchet mean. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, volume 119 of *Proceedings of Machine Learning Research*, pp. 6393–6403. PMLR, 2020. URL http://proceedings.mlr.press/v119/lou20a.html.

mary. Riemannax: Hardware-accelerated riemannian manifold optimization with jax, 2024. URL https://github.com/lv416e/riemannax.

Pascal Mettes, Mina Ghadimi Atigh, Martin Keller-Ressel, Jeffrey Gu, and Serena Yeung. Hyperbolic deep learning in computer vision: A survey. *Int. J. Comput. Vis.*, 132(9):3484–3508, 2024. doi: 10.1007/S11263-024-02043-5. URL https://doi.org/10.1007/s11263-024-02043-5.

Nina Miolane, Alice Le Brigant, Johan Mathe, Benjamin Hou, Nicolas Guigui, Yann Thanwerdas, Stefan Heyder, Olivier Peltre, Niklas Koep, Hadi Zaatiti, Hatem Hajri, Yann Cabanes, Thomas Gerald, Paul Chauchat, Christian Shewmake, Bernhard Kainz, Claire Donnat, Susan P. Holmes, and Xavier Pennec. Geomstats: A python package for riemannian geometry in machine learning. *CoRR*, abs/2004.04667, 2020. URL https://arxiv.org/abs/2004.04667.

Gal Mishne, Zhengchao Wan, Yusu Wang, and Sheng Yang. The numerical stability of hyperbolic representation learning. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (eds.), *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, volume 202 of *Proceedings of Machine Learning Research*, pp. 24925–24949. PMLR, 2023. URL https://proceedings.mlr.press/v202/mishne23a.html.

Yoshihiro Nagano, Shoichiro Yamaguchi, Yasuhiro Fujita, and Masanori Koyama. A wrapped normal distribution on hyperbolic space for gradient-based learning. In Kamalika Chaudhuri and Ruslan Salakhutdinov (eds.), *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*, volume 97 of *Proceedings of Machine Learning Research*, pp. 4693–4702. PMLR, 2019. URL http://proceedings.mlr.press/v97/nagano19a.html.

Maximilian Nickel and Douwe Kiela. Poincaré embeddings for learning hierarchical representations. In Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (eds.), *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long*

Beach, CA, USA, pp. 6338–6347, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/59dfa2df42d9e3d41f5b02bfc32229dd-Abstract.html.

Avik Pal, Max van Spengler, Guido Maria D'Amely di Melendugno, Alessandro Flaborea, Fabio Galasso, and Pascal Mettes. Compositional entailment learning for hyperbolic vision-language models. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. URL https://openreview.net/forum?id=3i13Gev2hV.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Z. Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d'Alché-Buc, Emily B. Fox, and Roman Garnett (eds.), *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pp. 8024–8035, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html.

Rik Sarkar. Low distortion delaunay embedding of trees in hyperbolic plane. In Marc J. van Kreveld and Bettina Speckmann (eds.), *Graph Drawing - 19th International Symposium, GD 2011, Eindhoven, The Netherlands, September 21-23, 2011, Revised Selected Papers*, volume 7034 of *Lecture Notes in Computer Science*, pp. 355–366. Springer, 2011. doi: 10.1007/978-3-642-25878-7_34. URL https://doi.org/10.1007/978-3-642-25878-7_34.

Xianglong Shi, Ziheng Chen, Yunhan Jiang, and Nicu Sebe. Intrinsic lorentz neural network. *CoRR*, abs/2602.23981, 2026. doi: 10.48550/ARXIV.2602.23981. URL https://doi.org/10.48550/arXiv.2602.23981.

Ryohei Shimizu, Yusuke Mukuta, and Tatsuya Harada. Hyperbolic neural networks++. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. URL https://openreview.net/forum?id=Ec85b0tUwbA.

Ondrej Skopek, Octavian-Eugen Ganea, and Gary Bécigneul. Mixed-curvature variational autoencoders. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020. URL https://openreview.net/forum?id=S1g6xeSKDS.

Abraham Albert Ungar. *A Gyrovector Space Approach to Hyperbolic Geometry*. Synthesis Lectures on Mathematics & Statistics. Morgan & Claypool Publishers, 2009. ISBN 978-3-031-01268-6. doi: 10.2200/S00175ED1V01Y200901MAS004. URL https://doi.org/10.2200/S00175ED1V01Y200901MAS004.

Saiteja Utpala, Andi Han, Pratik Jawanpuria, and Bamdev Mishra. Rieoptax: Riemannian optimization in JAX. *CoRR*, abs/2210.04840, 2022. doi: 10.48550/ARXIV.2210.04840. URL https://doi.org/10.48550/arXiv.2210.04840.

Robert van der Klis, Ricardo Chávez Torres, Max van Spengler, Yuhui Ding, Thomas Hofmann, and Pascal Mettes. Fast and geometrically grounded lorentz neural networks. *CoRR*, abs/2601.21529, 2026. doi: 10.48550/ARXIV.2601.21529. URL https://doi.org/10.48550/arXiv.2601.21529.

Max van Spengler, Erwin Berkhout, and Pascal Mettes. Poincaré resnet. In *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023*, pp. 5396–5405. IEEE, 2023a. doi: 10.1109/ICCV51070.2023.00499. URL https://doi.org/10.1109/ICCV51070.2023.00499.

Max van Spengler, Philipp Wirth, and Pascal Mettes. Hypll: The hyperbolic learning library. In Abdulmotaleb El Saddik, Tao Mei, Rita Cucchiara, Marco Bertini, Diana Patricia Tobon Vallejo, Pradeep K. Atrey, and M. Shamim Hossain (eds.), *Proceedings of the 31st ACM International Conference on Multimedia, MM 2023, Ottawa, ON, Canada, 29 October 2023- 3 November 2023*, pp. 9676–9679. ACM, 2023b. doi: 10.1145/3581783.3613462. URL https://doi.org/10.1145/3581783.3613462.

Menglin Yang, Harshit Verma, Delvin Ce Zhang, Jiahong Liu, Irwin King, and Rex Ying. Hypformer: Exploring efficient transformer fully in hyperbolic space. In Ricardo Baeza-Yates and Francesco Bonchi (eds.), *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2024, Barcelona, Spain, August 25-29, 2024*, pp. 3770–3781. ACM, 2024. doi: 10.1145/3637528.3672039. URL https://doi.org/10.1145/3637528.3672039.

Daiki Yoshikawa and Takashi Matsubara. Phyclip: $_1$-product of hyperbolic factors unifies hierarchy and compositionality in vision-language representation learning. *CoRR*, abs/2510.08919, 2025. doi: 10.48550/ARXIV.2510.08919. URL https://doi.org/10.48550/arXiv.2510.08919.

Kisung You. GeoJAX: A JAX-native toolkit for Riemannian geometry, manifold optimization, and manifold-valued statistics and machine learning, 2026. URL https://github.com/kisungyou/geojax.

A Layer Inventory

This appendix enumerates all 76 public exports of `hyperbolix.nn_layers` (the `__all__` list of `hyperbolix/nn_layers/__init__.py`, hyperbolix v1.3.0), grouped into ten families. "Type" distinguishes an `nnx.Module` (stateful, owns parameters/buffers) from a plain JAX function. Manifold abbreviations: Hyp = Hyperboloid (Lorentz) model, Poin = Poincaré ball, PV = Proper Velocity.

A.1 Linear

Export	Manifold(s)	Type	Source
`HTCLinear`	Hyp	Module	(Yang et al., 2024)
`FGGLinear`	Hyp	Module	(van der Klis et al., 2026)
`HypLinearHyperboloidPLFC`	Hyp	Module	(Shi et al., 2026)
`HypLinearHyperboloidFHCNN`	Hyp	Module	(Bdeir et al., 2024)
`HypLinearHyperboloidFHNN`	Hyp	Module	(Chen et al., 2022)
`HypLinearHyperboloidBusemann`	Hyp	Module	(Chen et al., 2026a)
`HypLinearPoincarePP`	Poin	Module	(Shimizu et al., 2021)
`HypLinearPoincare`	Poin	Module	(Ganea et al., 2018)
`HypLinearPoincareBusemann`	Poin	Module	(Chen et al., 2026a)
`HypLinearPV`	PV	Module	(Chen et al., 2026b)
`busemann_fc_poincare_output`	Poin	function	(Chen et al., 2026a)

Table 3: Linear layers (11 exports: ten layer classes and one function). `HypLinearPoincare` and `HypRegressionPoincare` (below) are the only layers in the library whose weights are manifold-valued and require a Riemannian optimizer; every other layer in this table trains with plain `optax.adam`.

A.2 Convolution

Export	Manifold(s)	Type	Source
`HypConv2DHyperboloid`	Hyp	Module	(Bdeir et al., 2024)
`HypConv2DHyperboloidFHNN`	Hyp	Module	(Chen et al., 2022)
`HypConv2DHyperboloidILNN`	Hyp	Module	(Shi et al., 2026)
`FGGConv2D`	Hyp	Module	(van der Klis et al., 2026)
`LorentzConv2D`	Hyp	Module	(He et al., 2025c)
`HypConv2DPoincare`	Poin	Module	(Shimizu et al., 2021; van Spengler et al., 2023a)
`HypConv2DPV`	PV	Module	(Chen et al., 2026b)

Table 4: Convolutional layers (7 exports). `LorentzConv2D` is kept for reproduction of the LResNet convolutional stack; it is HRC-based (Euclidean conv on the spatial components, time reconstructed from the hyperboloid constraint) and is superseded by `HypConv2DHyperboloid` for new work.

A.3 Attention

Export	Manifold(s)	Type	Source
`HyperbolicLinearAttention`	Hyp	Module	(Yang et al., 2024)
`HyperbolicSoftmaxAttention`	Hyp	Module	(Yang et al., 2024)
`HyperbolicFullAttention`	Hyp	Module	(Yang et al., 2024)
`focus_transform`	Hyp	function	(Yang et al., 2024)

Table 5: Attention layers (4 exports: three attention layers and one shared function). `HyperbolicLinearAttention` is $O(N)$ (kernel-trick linear attention with the focus function of `focus_transform`); `HyperbolicSoftmaxAttention` and `HyperbolicFullAttention` are $O(N^2)$, operating in the spatial domain and on full Lorentzian inner products respectively.

A.4 Normalization & Dropout

Export	Manifold(s)	Type	Source
`HyperboloidGyroBatchNorm`	Hyp	Module	(Chen et al., 2025b)
`HyperboloidGyroRMSNorm`	Hyp	Module	(Chen et al., 2025b)
`PoincareGyroRMSNorm`	Poin	Module	(Chen et al., 2025b)
`ProperVelocityGyroBatchNorm`	PV	Module	(Chen et al., 2025b)
`ProperVelocityGyroRMSNorm`	PV	Module	(Chen et al., 2025b)
`PoincareBatchNorm2D`	Poin	Module	(van Spengler et al., 2023a)
`HRCLayerNorm`	Hyp	Module	(Yang et al., 2024)
`HRCRMSNorm`	Hyp	Module	(Yang et al., 2024)
`HRCBatchNorm`	Hyp	Module	(Yang et al., 2024)
`HRCDropout`	Hyp	Module	(Yang et al., 2024)
`FGGMeanOnlyBatchNorm`	Hyp	Module	(van der Klis et al., 2026)
`frechet_variance`	any Manifold	function	(van Spengler et al., 2023a)

Table 6: Normalization and dropout layers (12 exports: ten normalization layers, one dropout layer, and one shared function). `*GyroBatchNorm`/`*GyroRMSNorm` are intrinsic (gyrovector) normalizers operating directly on manifold points; `HRC*` normalizers wrap a Euclidean `flax.nnx` layer on the spatial components via the Hypformer HRC pattern. `frechet_variance` is manifold-generic (used by both `PoincareBatchNorm2D` and the Gyro-BatchNorm family) and takes any object satisfying the `Manifold` protocol.

A.5 Residual, Pooling & Primitives

Export	Manifold(s)	Type	Source
`LorentzResidual`	Hyp	Module	(He et al., 2025c)
`lorentz_residual`	Hyp	function	(He et al., 2025c)
`lorentz_scale`	Hyp	function	(He et al., 2025c)
`lorentz_midpoint`	Hyp	function	(He et al., 2025a)
`hyp_avg_pool2d`	Hyp	function	(Yang et al., 2024)
`hyp_flatten2d`	Hyp	function	(Shi et al., 2026)
`hrc`	Hyp	function	(Yang et al., 2024)
`htc`	Hyp	function	(Yang et al., 2024)
`extract_patches`	Hyp	function	N/A
`hcat_ambient_dim`	Hyp	function	N/A
`build_spacelike_V`	Hyp	function	(van der Klis et al., 2026)
`spatial_to_hyperboloid`	Hyp	function	(Yang et al., 2024)
`sinh_lift_to_hyperboloid`	Hyp	function	(Shi et al., 2026)
`poincare_midpoint`	Poin	function	(Ungar, 2009; van Spengler et al., 2023a)
`poincare_weighted_midpoint`	Poin	function	(Bu et al., 2025)

Table 7: Residual connections, pooling, and shared low-level primitives (15 exports). `extract_patches` and `hcat_ambient_dim` are shared infrastructure behind every hyperboloid conv layer (im2col-style patch extraction and the HCat/LogCat output-dimension formula) with no single-paper attribution. `lorentz_midpoint` generalizes `lorentz_residual` (two points) to a weighted midpoint over M points via the closed-form Lorentz centroid of (He et al., 2025a); it backs `HyperbolicFullAttention`'s aggregation and `HyperboloidGyroBatchNorm`'s batch-mean estimator. `poincare_midpoint/poincare_weighted_midpoint` implement the Einstein/gyromidpoint used by `PoincareBatchNorm2D` and the Poincaré VQ codebook EMA update (Bu et al., 2025).

A.6 Positional Encoding

Export	Manifold(s)	Type	Source
`HypformerPositionalEncoding`	Hyp	Module	(Yang et al., 2024)
`HyperbolicRoPE`	Hyp	Module	(He et al., 2025a)
`hope`	Hyp	function	(He et al., 2025a)

Table 8: Positional encoding (3 exports). `HypformerPositionalEncoding` is a learnable relative encoding (`HTCLinear` + fixed-weight Lorentzian residual); `hope/HyperbolicRoPE` is the deterministic, rotation-based Hyperbolic Rotary Positional Encoding.

A.7 Regression / MLR

Export	Manifold(s)	Type	Source
`HypRegressionHyperboloid`	Hyp	Module	(Bdeir et al., 2024)
`FGGLorentzMLR`	Hyp	Module	(van der Klis et al., 2026)
`HypRegressionHyperboloidBusemann`	Hyp	Module	(Chen et al., 2026a)
`HypRegressionPoincarePP`	Poin	Module	(Shimizu et al., 2021)
`HypRegressionPoincare`	Poin	Module	(Ganea et al., 2018)
`HypRegressionPoincareBusemann`	Poin	Module	(Chen et al., 2026a)
`HypRegressionPV`	PV	Module	(Chen et al., 2026b)

Table 9: Multinomial logistic regression / classification heads (7 exports). All but `HypRegressionPoincare` return Euclidean logits from a point-to-hyperplane (or, for the Busemann pair, point-to-horosphere) score and train with plain `optax.adam`.

A.8 Vector Quantization

Export	Manifold(s)	Type	Source
`HypVQEmbeddingPoincare`	Poin	Module	(Chen et al., 2025a)
`HypVQMLRPoincare`	Poin	Module	(Goswami et al., 2025)
`PoincareVQOutput`	Poin	NamedTuple	(Chen et al., 2025a; Goswami et al., 2025)

Table 10: Poincaré-ball vector quantization (3 exports). `HypVQEmbeddingPoincare` is an explicit EMA codebook (buffer, not a trainable parameter) with a copy-gradient straight-through estimator; `HypVQMLRPoincare` is an implicit codebook realized as the rows of a Poincaré MLR, selected via Gumbel-Softmax quantization-as-classification and trainable end to end with Euclidean Adam. `PoincareVQOutput` is the shared `NamedTuple` return type of both.

A.9 Activations

Export	Manifold(s)	Type	Source
`hrc_relu`	Hyp	function	(Yang et al., 2024)
`hrc_leaky_relu`	Hyp	function	(Yang et al., 2024)
`hrc_tanh`	Hyp	function	(Yang et al., 2024)
`hrc_swish`	Hyp	function	(Yang et al., 2024)
`hrc_gelu`	Hyp	function	(Yang et al., 2024)
`hyp_relu`	Hyp	function	(Bdeir et al., 2024)
`hyp_leaky_relu`	Hyp	function	(Bdeir et al., 2024)
`hyp_tanh`	Hyp	function	(Bdeir et al., 2024)
`hyp_swish`	Hyp	function	(Bdeir et al., 2024)
`hyp_gelu`	Hyp	function	(Bdeir et al., 2024)
`poincare_relu`	Poin	function	(van Spengler et al., 2023a)
`poincare_leaky_relu`	Poin	function	(van Spengler et al., 2023a)
`poincare_tanh`	Poin	function	(van Spengler et al., 2023a)

Table 11: Activation functions (13 exports). `hrc_*` are curvature-changing ($c_{\text{in}} \neq c_{\text{out}}$) convenience wrappers around `hrc`; `hyp_*` are the curvature-preserving special case ($c_{\text{in}} = c_{\text{out}}$), documented against the fully-hyperbolic-CNN reference. `poincare_*` implement $f_{\mathcal{P}} = \exp_0^c \circ f \circ \log_0^c$ (activation applied in the tangent space at the origin).

A.10 Hybrid / Interface

Export	Manifold(s)	Type	Source
`HyperPPFeatureScaling`	Euclidean $\to$ Poin/Hyp	Module	(Klein et al., 2025)

Table 12: Euclidean-to-hyperbolic boundary layer (1 export). `HyperPPFeatureScaling` is not itself on-manifold: it applies parameter-free RMSNorm, a Lipschitz activation, and $1/\sqrt{d}$ dimension scaling to Euclidean features immediately before `expmap_0`, optionally followed by a learned sigmoid-bounded rescale $\rho_{\max} = \operatorname{atanh}(\alpha)/\sqrt{c}$ (Hyper++, (Klein et al., 2025), Sec. 3.2) that keeps the first on-manifold point strictly inside the ball/off the boundary.

B Implemented Methods

Table 13 maps published methods to the hyperbolix components that implement them, gathered from module- and class-level docstrings across the package (not only `nn_layers`).

Method	hyperbolix components	Reference
Hyperbolic Neural Networks (HNN)	`HypLinearPoincare`, `HypRegressionPoincare`, `ManifoldParam` + `riemannian_adam`	(Ganea et al., 2018)
HNN++ (point-to-hyperplane FC/MLR)	`HypLinearPoincarePP`, `HypRegressionPoincarePP`, `HypConv2DPoincare`	(Shimizu et al., 2021)
Poincaré ResNet	`HypConv2DPoincare`, `PoincareBatchNorm2D`, `poincare_relu`, `HyperPPFeatureScaling`	(van Spengler et al., 2023a)
Fully Hyperbolic Neural Networks (FHNN)	`HypLinearHyperboloidFHNN`, `HypConv2DHyperboloidFHNN`	(Chen et al., 2022)
Fully hyperbolic CNN (HCat)	`HypConv2DHyperboloid`, `HypLinearHyperboloidFHCNN`, `hyp_*` activations, `HypRegressionHyperboloid`	(Bdeir et al., 2024)
Hypformer HRC (Hyperbolic Regularization Component)	`hrc`, `hrc_*` activations, `HRCLayerNorm`, `HRCRMSNorm`, `HRCBatchNorm`, `HRCDropout`, `LorentzConv2D`	(Yang et al., 2024)
Hypformer HTC (Hyperbolic Transformation Component)	`htc`, `HTCLinear`	(Yang et al., 2024)
Hypformer attention	`HyperbolicLinearAttention`, `HyperbolicSoftmaxAttention`, `HyperbolicFullAttention`, `focus_transform`	(Yang et al., 2024)
Hypformer positional encoding	`HypformerPositionalEncoding`	(Yang et al., 2024)
Fast & Geometrically Grounded LNN (FGG-LNN)	`FGGLinear`, `FGGConv2D`, `FGGLorentzMLR`, `FGGMeanOnlyBatchNorm`, `build_spacelike_V`	(van der Klis et al., 2026)
Intrinsic Lorentz NN (ILNN), point-to-hyperplane FC (PLFC)	`HypLinearHyperboloidPLFC`, `sinh_lift_to_hyperboloid`	(Shi et al., 2026)
ILNN LogCat convolution	`HypConv2DHyperboloidILNN`, `hyp_flatten2d`, `Hyperboloid.log_radius_concat`	(Shi et al., 2026)
Intrinsic Lorentz gyrovector operations	`Hyperboloid.addition`, `Hyperboloid.scalar_mul`	(Shi et al., 2026)
Proper Velocity Neural Networks (PVNN)	`HypLinearPV`, `HypConv2DPV`, `HypRegressionPV`, `ProperVelocity` manifold	(Chen et al., 2026b)
Lorentzian ResNet (LResNet)	`LorentzResidual`, `lorentz_residual`, `lorentz_scale`, `LorentzConv2D`	(He et al., 2025c)
Hyperbolic Busemann Networks	`HypLinearHyperboloidBusemann`, `HypLinearPoincareBusemann`, `HypRegressionHyperboloidBusemann`, `HypRegressionPoincareBusemann`	(Chen et al., 2026a)
Gyrogroup Batch Normalization (GyroBN)	`HyperboloidGyroBatchNorm`, `ProperVelocityGyroBatchNorm`, `*GyroRMSNorm` family	(Chen et al., 2025b)
Hyperbolic Rotary Positional Encoding (HOPE / HELM)	`hope`, `HyperbolicRoPE`, `lorentz_midpoint`	(He et al., 2025a)
Hyperbolic VQ-VAE, EMA codebook (HVQ-VAE)	`HypVQEmbeddingPoincare`, `poincare_weighted_midpoint`	(Chen et al., 2025a)
Hyperbolic VQ-VAE, quantization-as-classification (HyperVQ)	`HypVQMLRPoincare`	(Goswami et al., 2025)
κ-Stereographic model	`Stereographic` manifold class	(Bachmann et al., 2020)
HoroPCA	`hyperbolix.decomposition.horopca`	(Chami et al., 2021)
CO-SNE	`hyperbolix.decomposition.cosne`	(Guo et al., 2022)
Fréchet / Karcher mean	`hyperbolix.decomposition.frechet.frechet_mean`	(Lou et al., 2020)
Wrapped normal distribution	`wrapped_normal_hyperboloid`, `wrapped_normal_poincare`	(Nagano et al., 2019)
Riemannian Adam	`hyperbolix.optim.riemannian_adam`, `ManifoldParam`	(Bécigneul & Ganea, 2019)
Riemannian SGD	`hyperbolix.optim.riemannian_sgd`	(Bonnabel, 2013)
Product (mixed-curvature) manifolds	`ProductManifold`, per-factor `LearnableCurvature`	(Gu et al., 2019)
Poincaré embeddings (smoothened distance)	`Hyperboloid.dist/dist_0`, `VERSION_SMOOTHENED`	(Nickel & Kiela, 2017)
Gyrovector formalism	`Poincare/Hyperboloid/ProperVelocity addition`, `scalar_mul`	(Ungar, 2009)
Delta-hyperbolicity / Gromov product	`hyperbolix.utils.helpers.compute_hyperbolic_delta`	(Khrulkov et al., 2020)

Table 13: Implemented methods and the hyperbolix components realizing them (31 rows).

C The JAX Ecosystem

Table 14 documents the three JAX libraries with a focus similar to hyperbolix that provide some Riemannian or hyperbolic functionality, verified against their default-branch sources on 2026-08-03. None offers a general-purpose hyperbolic layer library. Rieoptax contains nine undocumented Poincaré/Lorentz layer classes (no convolutions) and has been inactive since October 2023. RiemannAX's hyperbolic manifolds exist only in unreleased development code. GeoJAX, created in July 2026 and currently under active development, implements both the Poincaré ball and hyperboloid models with a broad suite of Riemannian optimization methods. However, it contains no neural-network layers.

	Rieoptax	RiemannAX	GeoJAX
Stated focus	Riemannian optim., diff. privacy	Riemannian optim.	Riemannian geometry, optim., classical ML
Hyperbolic manifolds	P, H	P, H (unreleased)	P, H
Hyperbolic NN layers	9 undocumented classes, no conv	No (one manifold-agnostic linear)	No
Riemannian optimizers	SGD (Adam stubbed)	SGD, Adam, momentum	First- and second-order suite
Last activity (as of 2026-08)	Oct. 2023	Mar. 2026	Active
License	MIT (metadata only)	Apache-2.0	MIT

Table 14: JAX libraries with Riemannian or hyperbolic functionality, from their default-branch sources as of 2026-08-03. P: Poincaré ball, H: hyperboloid. Rieoptax lists MIT only in its package metadata; the repository ships no license file.

D The Manifold Protocol

`hyperbolix.manifolds.protocol.Manifold` is a `typing.Protocol` decorated `@runtime_checkable`: no concrete manifold class inherits from it, and `isinstance(x, Manifold)` succeeds for any object that exposes the right method signatures. This structural typing is what lets `ProductManifold` satisfy the protocol without a shared base class — it is a plain composition of factor manifolds that happens to expose the same 19-method surface (its `c` argument is a per-factor *sequence* rather than a scalar, which the protocol's `Curvature = ScalarCurvature | Sequence[ScalarCurvature]` union accommodates). `Poincare`, `Hyperboloid`, `ProperVelocity`, `Euclidean`, and `Stereographic` all conform the same way.

Table 15 lists the 19 required methods (the `dtype` field is an attribute, not a method, and is excluded from the count) with one-line semantics.

Method	Semantics
`_cast(x)`	Cast an input array to the manifold's compute dtype.
`proj(x, c)`	Project a point back onto the manifold's defining constraint.
`dist(x, y, c)`	Geodesic distance between two manifold points.
`dist_0(x, c)`	Geodesic distance from a point to the manifold origin.
`addition(x, y, c)`	Gyrovector (Möbius / Lorentz) addition of two manifold points.
`scalar_mul(r, x, c)`	Gyrovector scalar multiplication; scales the geodesic radius by r.
`expmap(v, x, c)`	Riemannian exponential map: move from x along tangent v to a manifold point.
`expmap_0(v, c)`	Exponential map at the origin (v assumed tangent there).
`logmap(y, x, c)`	Riemannian logarithmic map (inverse of `expmap`) from x to y.
`logmap_0(y, c)`	Logarithmic map at the origin.
`retraction(v, x, c)`	First-order approximation to `expmap`, used by first-order Riemannian optimizers.
`ptransp(v, x, y, c)`	Parallel-transport tangent vector v from the tangent space at x to that at y.
`ptransp_0(v, y, c)`	Parallel transport from the origin's tangent space to y's tangent space.
`tangent_inner(u, v, x, c)`	Riemannian inner product of two tangent vectors at x.
`tangent_norm(v, x, c)`	Riemannian norm of a tangent vector at x.
`egrad2rgrad(grad, x, c)`	Convert a Euclidean gradient into the Riemannian gradient at x.
`tangent_proj(v, x, c)`	Project an ambient vector onto the tangent space at x.
`is_in_manifold(x, c)`	Check whether x satisfies the manifold's constraint within `atol`.
`is_in_tangent_space(v, x, c)`	Check whether v is a valid tangent vector at x within `atol`.

Table 15: The 19 methods of the `Manifold` protocol. `dist`/`dist_0` additionally take a trailing static `version_idx: int` (manifolds with one implementation accept and ignore it); `is_in_manifold`/`is_in_tangent_space` take a trailing `atol: float | None = None`, resolved through `default_atol(dtype)` when `None`.

Table 16 lists the exact, distance-preserving isometry maps between the three point-set models shipped in `hyperbolix.manifolds.isometry_mappings`. Every off-diagonal pair has both directions implemented; none route through `logmap`/`expmap` (which would be lossy and slower). The Hyperboloid↔PV pair is a direct map (PV coordinates are exactly the space-like part of the hyperboloid point, with the time coordinate reconstructed from the Lorentz constraint); it is not the composition of the other two maps, which would pass through the Poincaré conformal factor and lose accuracy near the ball boundary.

Model pair	Map	Inverse map
Poincaré ↔ Hyperboloid	`poincare_to_hyperboloid`	`hyperboloid_to_poincare`
Poincaré ↔ ProperVelocity	`poincare_to_pv`	`pv_to_poincare`
Hyperboloid ↔ ProperVelocity (direct)	`hyperboloid_to_pv`	`pv_to_hyperboloid`

Table 16: Exact isometries between the three point-set manifold models (`hyperbolix.manifolds.isometry_mappings`). All three pairs are covered in both directions; the Hyperboloid↔ProperVelocity pair is a direct map rather than a composition through the Poincaré ball.

E Initialization Defaults

Table 17 lists the shipped default initialization for each layer family. Where a family does not reproduce its source paper's initialization, the default is scaled by fan-in or fan-out rather than set to a fixed bound.

Layer family	Default init	Rationale
Poincaré FC (`HypLinearPoincare`, `HypLinearPoincarePP`)	std $= 1/\sqrt{\text{fan_in}}$	Keeps first-layer output away from the ball boundary.
Poincaré regression / MLR (`HypRegressionPoincare`, `HypRegressionPoincarePP`)	std $= (2 \cdot \text{in} \cdot \text{out})^{-1/2}$	Matches (van Spengler et al., 2023a); unscaled $\mathcal{N}(0,1)$ row norms $\approx \sqrt{\text{in_dim}}$ overwhelm the MLR output scaling.
Poincaré conv (`HypConv2DPoincare`, `id_init=True`)	$0.5 \cdot \mathbf{I}$ (identity)	The 0.5 factor compensates the factor of 2 inside the HNN++ distance formula.
`HTCLinear` (+ wrappers, e.g. attention Q/K/V, `HypformerPositionalEncoding`)	fan-in uniform $U(\pm\sqrt{3/\text{in_features}})$	Per-layer input-Jacobian gain ≈ 1 (no nonlinearity in the `htc` tail); a fixed bound (e.g. the legacy 0.02) is width-dependent and contracts at large fan-in.
FHNN / FHCNN linear & conv (`HypLinearHyperboloidFHNN`, `HypLinearHyperboloidFHCNN`, `HypConv2DHyperboloidFHNN`)	$U(-0.02, 0.02)$	Matches (Chen et al., 2022; Bdeir et al., 2024) reference init (FHNN additionally zeroes the time column, giving tangent vectors at the origin).
PLFC / ILNN linear (`HypLinearHyperboloidPLFC`)	std $= 0.02$	Matches the (Shi et al., 2026) PLFC reference value.
ILNN conv (`HypConv2DHyperboloidILNN`)	std $= \sqrt{1/\text{out_spatial}}$ (fan-out, default)	Norm-preserving under the corrected LogCat digamma rescale; `kernel_init_std=0.02` restores the paper reference (tuned for the pre-fix LogCat and now strongly contractive).
PV hidden layers (`HypLinearPV`, `HypConv2DPV`)	He, $\sqrt{2/\text{fan_in}}$	Near the origin the PV output reduces to a Euclidean linear map, so standard He/ReLU variance-preservation analysis applies.
PV regression head (`HypRegressionPV`)	std $= 10^{-2}$	Matches the paper's `PVManifoldMLR.reset_parameters`, tuned for $O(1)$-variance input features immediately before a softmax/MSE loss.
FGG linear & conv, default (`FGGLinear`, `FGGConv2D`, `reset_params="fan_out"`)	std $= \sqrt{1/\text{out_spatial}}$	Norm-preserving; a deliberate RL-tuned deviation from the (van der Klis et al., 2026) classification reference (`"eye"` init + `init_bias=0.5`), which assumes a following BatchNorm.
Hyperboloid HCat conv, default (`HypConv2DHyperboloid`, `reset_params="default"`)	$U(-0.02, 0.02)$	Unchanged FHCNN-style init; alternative `"fan_in"` uses std $= \sqrt{1/\text{hcat_out_ambient_dim}}$.
Busemann FC (`HypLinearHyperboloidBusemann`, `HypLinearPoincareBusemann`)	weight-norm split, direction std $= (2 \cdot \text{in} \cdot \text{out})^{-1/2}$, log_scale $=$ log‖kernel row‖	Reproduces the reference `weight_g`/`weight_v` split; matches the HNN++ FC scale convention.
Busemann MLR (`HypRegressionHyperboloidBusemann`, `HypRegressionPoincareBusemann`)	direction std $= \text{in_spatial}^{-1/2}$, log_scale $=$ log‖kernel row‖	Same weight-norm split as the FC layers, matching the paper's MLR scale.

Table 17: Shipped initialization defaults by layer family. Also the reference values for custom reimplementations.

Across every family above, the shipped default is chosen so the per-layer input-Jacobian gain sits near 1. This single design constraint has an asymmetric failure mode on either side. A generic Euclidean init (He/Xavier, or a fixed bound calibrated for a different fan-in) that overshoots gain 1 pushes the first-layer output toward the Poincaré boundary or far up the hyperboloid, producing `NaN` losses within a few steps — loud and easy to diagnose. An init that undershoots gain 1 fails silently: for a stack of linear-in-the-matmul layers the gain compounds as g^{depth}, and once pairwise output distances fall below the float32 resolution of the distance computation itself (near the origin, $\texttt{acosh}(1 + c\,d^2/2)$ quantizes to exactly zero once $c\,d^2/2 < \varepsilon_{f32} \approx 1.19 \times 10^{-7}$), the stack becomes a constant map: gradients are ≈ 0 from step zero, with no `NaN` or warning to flag it. Both failure modes have been observed in this codebase's own history (documented in the `HTCLinear` and `HypConv2DHyperboloidILNN` docstrings) and motivate replacing library-wide fixed-bound inits with fan-in/fan-out-scaled ones.

F Numerical Conventions

`MIN_NORM` and its finite-VJP rationale. `hyperbolix.utils.math_utils.MIN_NORM = 1e-15` is the library-wide gradient-safety floor for norms and denominators, stabilizing gradients in two ways. First, `safe_norm` (a max-scaled reduction floored at `MIN_NORM` via `floor_at`) has a finite gradient at $x = 0$. In contrast, a plain `jnp.linalg.norm` evaluates the derivative as $x/\|x\| = 0/0 = \text{NaN}$. Second, `maximum(denom, MIN_NORM)` keeps a division finite when a denominator collapses. 10^{-15} is small enough to be invisible next to any float32 or float64 quantity the library works with (float64 machine epsilon is 2.2×10^{-16}).

`tanh` input/output clamping. `hyperbolix.utils.math_utils.tanh` clips its *input* to $\pm 0.5 \log(2/\varepsilon)$ ($\approx \pm 8.3$ for float32, $\approx \pm 18.4$ for float64) and additionally clips its *output* to $\pm(1 - 10\varepsilon)$. Both clips are needed: the input clip is the analytic point where $1 - \tanh(x)$ reaches ε, but XLA's float32 `tanh` saturates to *exactly* 1.0 slightly before that bound (around $x = 8$), which the input clip alone does not prevent. An output of exactly 1.0 would make a downstream `atanh` call hit its pole and inject a `NaN` cotangent into the VJP that no later `jnp.where` can remove. The output clip matches `atanh`'s own domain guard, so `atanh(tanh(x))` can never reach the singularity.

Tolerance conventions. Table 18 gives the two absolute-tolerance conventions used throughout the test suite and library.

Convention	float32	float64
`default_atol(dtype)` $= \sqrt{\varepsilon_{\text{dtype}}}$ (manifold membership)	3.45×10^{-4}	1.49×10^{-8}
Suite-wide dtype fixture `atol`/`rtol` (`tests/conftest.py`)	4×10^{-3}	1×10^{-7}

Table 18: Default absolute tolerances. `default_atol` is the resolution used by every `is_in_manifold`/`is_in_tangent_space` call left at `atol=None`; an explicit `atol` always wins and is never floored or clamped.

`default_atol` is $\sqrt{\varepsilon}$ because the constraint residuals it checks—$\langle x, x \rangle_{\mathcal{L}} + 1/c$ on the hyperboloid, $c\|x\|^2 - 1$ on the ball—are differences of same-magnitude sums that lose roughly half the mantissa. Both residuals are dimensionless, so a single tolerance works at every curvature. The tolerance is absolute and calibrated for points near the origin: a genuinely on-manifold point far from the origin can fail the default check (float64 holds to geodesic distance $\sim$10, float32 to $\sim$7), which is the regime where the library recommends float64.

Dtype guidance. Table 19 summarizes when float32 is sufficient.

Geodesic distance from origin	float32 accuracy	Recommendation
$d < 3$	excellent ($< 0.01\%$ error)	float32
$3 \leq d < 5$	good ($< 0.1\%$ error)	float32
$5 \leq d < 10$	moderate ($< 3\%$ error)	float64 for critical ops
$d \geq 10$	poor ($> 3\%$ error)	float64 required

Table 19: Poincaré ball precision requirements by geodesic distance from the origin. The hyperboloid works for float32 much further out due to its cancellation-free formulas.

Storage and compute precision are separate, orthogonal knobs. *Compute* precision is the dtype in which manifold operations (`dist`, `expmap`, `logmap`, ...) run, set by the manifold instance's `dtype` attribute (e.g. `Poincare(dtype=jnp.float64)`); manifold methods cast their array arguments to this dtype on entry. *Storage* precision is the dtype of a layer's trainable parameters and persistent state (batch-norm running statistics, VQ codebooks), set independently by the `param_dtype` constructor argument on every NN layer (default `jnp.float32`). A float32-stored parameter that enters a float64 manifold computation is promoted

for that computation only; the stored parameter itself is untouched. The recommended high-precision recipe is therefore float64 compute with float32 storage — full geometric precision where it matters, at half the parameter/optimizer-state memory.

G Test-Suite Structure

At v1.3.0 the suite comprises 55 `test_*.py` files (two `conftest.py` fixture files and two `__init__.py` files bring the directory to 59 `.py` files) totaling 1,038 top-level `def test_*` functions. Under `pytest` parametrization (dtype × curvature × layer-spec axes) this expands to 4,446 collected test items.

G.1 The 18-property layer contract

`tests/nn_layers/test_layer_contract.py` replaces the former per-layer six-slot boilerplate (copy-pasted once per layer class across a dozen files) with a single `LayerSpec` table and five parametrized contract tests, run over every layer in the table. Enumerated as 18 individually checkable properties:

1. Forward output shape matches the declared `out_shape`.
2. Forward output dtype matches the input dtype (float32).
3. Forward output is entirely finite.
4. Forward output lies on the manifold (`is_in_manifold`), for every manifold-valued layer.
5. `nnx.jit`-compiled forward output shape matches the eager result.
6. `nnx.jit`-compiled forward output values match the eager result (`allclose`, atol = rtol = 10^{-5}).
7. Output is finite at a non-unit curvature $c = 0.5$.
8. Output stays on the manifold at $c = 0.5$.
9. Output is finite at a non-unit curvature $c = 2.0$.
10. Output stays on the manifold at $c = 2.0$.
11. Loss (sum of squared outputs) is finite.
12. Every declared gradient path (`kernel`, `bias`, and any layer-specific extra parameter such as `scale`, `log_scale`, `gyro_bias`) is finite, at both float32 and float64.
13. The primary parameter's gradient has at least one nonzero entry (non-trivial signal).
14. `nnx.jit`-compiled gradient equals the eager gradient for every declared path (tolerance scaled by the largest gradient entry).
15. The `input_space="tangent"` forward output has the correct shape.
16. The `input_space="tangent"` branch agrees exactly with an explicit `expmap_0` followed by the `input_space="manifold"` branch (atol = 10^{-9}, float64).
17. Convolutional output spatial size follows the declared padding/stride arithmetic, across kernel size × padding × stride combinations.
18. Convolutional output is entirely finite under every kernel/padding/stride combination.

G.2 Oracle taxonomy

Every numeric assertion in the suite is checked against one of three independent oracle sources — never against another hyperbolic-geometry library:

- **Hand-derived closed forms**, worked out independently of the implementation and, where cancellation is a concern, evaluated at extended precision (e.g. `numpy.longdouble`, 80-bit on x86) so the oracle's own rounding error sits several decades below the assertion tolerance.
- **NumPy/SciPy transcriptions** of the source papers' formulas, built from primitive NumPy operations rather than calling into any hyperbolic-geometry package (e.g. `scipy.stats.multivariate_normal` for the flat-curvature limit of the wrapped normal, or a from-scratch NumPy geodesic frame for the log-probability integration test).
- **Finite differences / independent autodiff**, e.g. cross-checking a closed-form log-Jacobian against `jax.jacfwd` of the sampling map itself, so a bug shared between the closed form and its *use* would still be caught by the independent differentiation path.

G.3 Wrapped-normal log-probability: four-direction pinning

`tests/test_wrapped_normal.py` pins the wrapped-normal `log_prob` value (not just its shape/finiteness) from four independent directions, for both the Hyperboloid and Poincaré implementations:

1. **Quadrature integrates to one.** $\int \exp(\texttt{log_prob})\, dV = 1$ against the Riemannian volume element of $\mathbb{H}^2$ in geodesic polar coordinates (Gauss-Legendre in radius, trapezoid in angle), at three curvatures and a non-origin mean.
2. **Change-of-variables vs. `jacfwd`.** `log_prob` is checked against the pushforward density of the sampling map itself: the tangent-space Gaussian density divided by $|\det \partial T/\partial v|$, with the Jacobian obtained from `jax.jacfwd` of the sampling map — never from the library's own closed-form log-determinant.
3. **Flat limit vs. SciPy multivariate normal.** At $c = 10^{-4}$, both models must reduce to a Euclidean Gaussian in Riemannian coordinates, matching `scipy.stats.multivariate_normal.logpdf` to 10^{-3} (the residual is the expected $O(c\, r^2)$ curvature correction).
4. **Isometry cross-check.** The Hyperboloid and Poincaré log-densities must agree *pointwise, with no correction factor* at corresponding points related by `hyperboloid_to_poincare` — since that map is an isometry, it carries the Riemannian volume measure over unchanged.

H Measured Accuracy, Stability, and Speed

This appendix reports the measurements behind Table 20, Figures 3, 4 and 5, and the speed and cancellation tables that follow. Every number comes from the released wheel hyperbolix 1.3.0, installed from PyPI into a fresh environment per script rather than imported from a source checkout, so the tables describe what a user installs. The baselines are geoopt 0.5.1 (Kochurov et al., 2020) and HypLL 0.1.1 (van Spengler et al., 2023b) on PyTorch 2.13.0. The six measurement scripts write JSON, and five rendering scripts turn that JSON into every table and figure without touching any library.

H.1 Common protocol

Oracles. Every accuracy number is a deviation from a NumPy float64 transcription of a published closed form, evaluated on points that are themselves sampled in NumPy. No oracle calls hyperbolix, geoopt, or HypLL; hyperboloid points are lifted with the NumPy $\exp_0$ oracle rather than with any library map. A deviation is therefore always library against reference, never library against itself.

Timing. All speed measurements use $D = 32$ spatial dimensions, curvature $c = 1$, and float32. Operands are generated once in NumPy and handed to every library, so the columns of a row time the same numbers, not merely the same shapes. Every timed callable synchronizes internally (`jax.block_until_ready` for JAX, `torch.cuda.synchronize` for PyTorch on GPU); the timer holds only the clock. A cell is the median over 20 timed repeats after 5 warm-up calls. HypLL's `ManifoldTensor` wrappers are constructed outside the timed region. The hyperbolix eager column runs `jax.vmap` over the single-point API without `jax.jit`, which re-traces on every call; it measures host-side dispatch, and the jit column is the kernel comparison.

Launches and brackets. We call one process invocation of the speed script a *launch*. Every speed table aggregates five sequential launches: a cell is the median of the five per-launch medians, and the bracket beneath it is the minimum and maximum of those five medians. All five launches use the same seed and therefore identical operands, so a bracket bounds launch-to-launch machine and compilation state, not sampling variation. Where we report that one library is faster "in all five launches", the ordering held across all launches, and the two brackets do not overlap.

Absent cells. An n/a cell is structurally absent rather than unmeasured, with one exception. HypLL ships no hyperboloid model, so its hyperboloid cells do not exist; geoopt ships no neural-network layers, so it has no layer rows. The exception is the HypLL compiled cell of the Poincaré linear row: HypLL's `HLinear` holds a `ManifoldParameter` whose `__torch_function__` raises when Dynamo probes it, so `torch.compile` fails on that layer while succeeding on all five HypLL primitives. Two rows are not projection-matched: HypLL's `mobius_add` does not project its result back into the ball, whereas geoopt's and hyperbolix's do.

H.2 Deviation from independent oracles

Table 20 measures hyperbolix alone against the oracles. For every manifold, operation class, and dtype, the script samples 2 048 points per cell over dimensions $\{2, 10\}$ and curvatures $\{0.3, 1.0, 2.5\}$; the κ-stereographic model adds the spherical branch $c = -1$, on which tangent norms are capped at $0.9 \cdot \pi/2$, below the pole of tan. Each table row aggregates the maximum and the median absolute deviation over its whole curvature and dimension grid. The layer-forward row (MLR) exists for the hyperboloid and proper-velocity manifolds, which are the two on which the library ships a closed-form multinomial regression head with a published oracle.

Every float32 cell lies below 4.6×10^{-6} and every float64 cell at or below 3.5×10^{-14}, so all cells satisfy the library's membership tolerance $\sqrt{\varepsilon}$ (3.45×10^{-4} in float32, 1.49×10^{-8} in float64) with margin. The deviations are absolute, which matters for reading the largest float32 cell: stereographic distance, at 4.6×10^{-6}, is only $1.15\times$ the next-largest float32 cell (Poincaré distance, 4.0×10^{-6}). The oracle itself is a fresh derivation; it satisfies the tangency invariant $\langle y, \mathrm{PT}_{x \to y} v \rangle_{\mathcal{L}} = 0$ to 2.8×10^{-14} over the whole grid.

Table 20: Maximum and median *absolute* deviation between hyperbolix 1.3.0 operations and independent NumPy float64 oracles, over 2 048 sampled points per (curvature, dimension) cell, curvatures $\{0.3, 1.0, 2.5\}$ (plus $c = -1$ for the stereographic model) and dimensions $\{2, 10\}$. Every cell lies below the membership tolerance $\sqrt{\varepsilon}$ of its dtype. The largest float32 cell (stereographic distance, 4.6×10^{-6}) is only $1.15\times$ the next-largest cell (Poincaré distance). Layer-forward (MLR) rows exist for the hyperboloid and proper-velocity manifolds only. Script: `scripts/oracle_deviation.py`, CPU backend.

		float32		float64	
Manifold	Operation class	max	median	max	median
Poincaré	dist	4.0×10^{-6}	1.6×10^{-7}	7.1×10^{-15}	3.3×10^{-16}
Poincaré	expmap/logmap	2.7×10^{-7}	5.8×10^{-9}	4.4×10^{-16}	0
Poincaré	ptransp	4.3×10^{-7}	4.4×10^{-9}	6.7×10^{-16}	0
Hyperboloid	dist	1.1×10^{-6}	7.6×10^{-8}	3.5×10^{-14}	2.2×10^{-16}
Hyperboloid	expmap/logmap	1.4×10^{-6}	8.6×10^{-9}	4.7×10^{-15}	0
Hyperboloid	ptransp	1.4×10^{-6}	7.2×10^{-9}	3.6×10^{-15}	0
Hyperboloid	MLR forward	2.0×10^{-6}	4.1×10^{-8}	3.6×10^{-15}	1.0×10^{-17}
ProperVelocity	dist	1.3×10^{-6}	9.6×10^{-8}	8.2×10^{-15}	2.2×10^{-16}
ProperVelocity	expmap/logmap	1.1×10^{-6}	1.3×10^{-8}	2.7×10^{-15}	0
ProperVelocity	ptransp	1.4×10^{-6}	1.0×10^{-8}	1.8×10^{-15}	0
ProperVelocity	MLR forward	1.9×10^{-6}	6.3×10^{-8}	2.7×10^{-15}	0
Stereographic	dist	4.6×10^{-6}	1.2×10^{-7}	1.1×10^{-14}	2.2×10^{-16}
Stereographic	expmap/logmap	3.8×10^{-6}	6.0×10^{-9}	6.2×10^{-15}	0
Stereographic	ptransp	4.6×10^{-7}	4.6×10^{-9}	4.4×10^{-16}	0

H.3 Cross-library accuracy on the Poincaré ball

Table 21 applies the same oracles to all three libraries. It is restricted to the Poincaré ball because that is the only model all three implement. For each of five operations, three curvatures $\{0.3, 1.0, 2.5\}$, two dimensions $\{2, 10\}$, and both dtypes, the script samples 2 048 points, evaluates each library and the oracle on the same points, and records the maximum absolute deviation; the table reports the worse of the two dimensions.

In float32 the three libraries are interchangeable: across the 90 (operation, curvature, dimension, library) cells before the dimension collapse, deviations lie between 6.3×10^{-8} and 4.8×10^{-6}, and within any one (operation, curvature, dimension) cell the three libraries differ by at most $2.2\times$ (median $1.24\times$). The float64 half separates them, and the separation is about curvature storage rather than arithmetic. HypLL stores its curvature as float32 (`Curvature.__init__` casts to `torch.float32`, and `.double()` cannot recover the lost digits), so at $c = 0.3$ every HypLL float64 table cell floors between 1.8×10^{-8} and 8.8×10^{-8} (over the underlying grid, before the dimension collapse, 1.6×10^{-8} to 8.8×10^{-8}), while $c = 1$ and $c = 2.5$, which are exactly representable in float32, reach machine precision. geoopt keeps the inverse softplus of c as a parameter and passes a Python float through `torch.as_tensor`, which also lands in float32; the natural call `PoincareBall(c=0.3)` floors at $\sim 10^{-7}$, worse than HypLL (measured in a side probe of the accuracy audit; the released grid contains only the typed-tensor call). The geoopt column reaches machine precision only because the script passes an explicitly typed float64 tensor, an escape hatch HypLL lacks. hyperbolix takes the curvature as a call-time scalar in the working dtype and has no such floor.

Table 21: Maximum absolute deviation from the NumPy float64 oracle on the Poincaré ball, for geoopt 0.5.1, HypLL 0.1.1, and hyperbolix 1.3.0, over 2 048 sampled points per cell and the worse of dimensions $\{2, 10\}$. In float32 the three libraries agree to within 2.2× in every cell. In float64, HypLL's cells at $c = 0.3$ floor at $\sim 10^{-8}$ because HypLL stores the curvature in float32; geoopt reaches machine precision only when given an explicitly typed float64 tensor rather than a Python float. Script: `scripts/crosslib_accuracy.py`, CPU backend.

		float32			float64		
Operation	c	geoopt	HypLL	hyperbolix	geoopt	HypLL	hyperbolix
$d(x, y)$	0.3	4.8×10^{-6}	3.8×10^{-6}	4.0×10^{-6}	3.6×10^{-14}	8.8×10^{-8}	8.8×10^{-15}
$d(x, y)$	1.0	1.7×10^{-6}	1.6×10^{-6}	1.7×10^{-6}	6.7×10^{-15}	3.1×10^{-15}	3.1×10^{-15}
$d(x, y)$	2.5	1.2×10^{-6}	1.2×10^{-6}	1.5×10^{-6}	3.3×10^{-15}	2.4×10^{-15}	2.9×10^{-15}
$\exp_0(v)$	0.3	3.5×10^{-7}	2.5×10^{-7}	2.6×10^{-7}	3.1×10^{-15}	3.0×10^{-8}	6.7×10^{-16}
$\exp_0(v)$	1.0	1.0×10^{-7}	1.1×10^{-7}	1.5×10^{-7}	5.6×10^{-16}	3.3×10^{-16}	2.2×10^{-16}
$\exp_0(v)$	2.5	9.8×10^{-8}	8.1×10^{-8}	9.8×10^{-8}	2.2×10^{-16}	2.2×10^{-16}	2.2×10^{-16}
$\log_0(x)$	0.3	5.1×10^{-7}	2.6×10^{-7}	2.7×10^{-7}	2.0×10^{-15}	1.8×10^{-8}	4.4×10^{-16}
$\log_0(x)$	1.0	1.3×10^{-7}	1.3×10^{-7}	1.4×10^{-7}	5.6×10^{-16}	2.2×10^{-16}	2.2×10^{-16}
$\log_0(x)$	2.5	9.8×10^{-8}	7.7×10^{-8}	7.7×10^{-8}	2.2×10^{-16}	1.7×10^{-16}	1.1×10^{-16}
$x \oplus_c y$	0.3	3.9×10^{-7}	4.9×10^{-7}	4.8×10^{-7}	9.0×10^{-16}	3.1×10^{-8}	8.9×10^{-16}
$x \oplus_c y$	1.0	2.2×10^{-7}	2.2×10^{-7}	2.7×10^{-7}	2.2×10^{-16}	2.2×10^{-16}	5.6×10^{-16}
$x \oplus_c y$	2.5	1.3×10^{-7}	1.3×10^{-7}	1.6×10^{-7}	1.7×10^{-16}	1.7×10^{-16}	4.4×10^{-16}
$PT_{0 \to x}(v)$	0.3	6.3×10^{-7}	4.3×10^{-7}	4.3×10^{-7}	8.9×10^{-16}	7.0×10^{-8}	4.4×10^{-16}
$PT_{0 \to x}(v)$	1.0	2.2×10^{-7}	2.2×10^{-7}	2.2×10^{-7}	3.3×10^{-16}	3.3×10^{-16}	2.2×10^{-16}
$PT_{0 \to x}(v)$	2.5	1.5×10^{-7}	1.5×10^{-7}	1.6×10^{-7}	2.2×10^{-16}	2.2×10^{-16}	2.2×10^{-16}

H.4 Round-trip stability

Figures 3 and 4 share one setup: dimension 8, $c = 1$, 512 tangent vectors per radius, and 40 norms spaced logarithmically from 10^{-3} to 20. The x axis is the tangent-vector norm (which equals geodesic distance on the hyperboloid and half the geodesic distance on the Poincaré ball). Both ran on the CPU backend, with relative errors computed in float64.

We define the *breakdown radius* as the first grid point where the float32 median relative error exceeds $\sqrt{\varepsilon_{32}}$. On the Poincaré ball it is $r = 3.38$ for geoopt and HypLL and $r = 7.24$ for hyperbolix. This difference is due to a design choice about the projection margin: hyperbolix projects onto a ball of radius $1/\sqrt{c} - \varepsilon^{0.75}$, and $\operatorname{artanh}(1 - \varepsilon_{32}^{0.75}) = 6.33$ is where the round trip must fail. On the hyperboloid, geoopt already exceeds the threshold at $r = 10^{-3}$ with a median of 2.3×10^{-2} against 1.1×10^{-7} for hyperbolix. hyperbolix stays below it through $r = 20$.

The hyperbolix-only figure adds the proper-velocity and κ-stereographic models, both dtypes, and the distance variants selected through `version_idx`. Its float32 Poincaré curve has a knee at tangent norm $r = 7.24$ (geodesic distance 14.5), consistent with the margin above; in float64 the same round trip is still 2.8×10^{-8} at $r = 12.0$. The proper velocity to hyperboloid isometry round trip is exactly zero in both dtypes by construction: the map to the hyperboloid concatenates a recomputed time coordinate, and the map back slices it off. Near the origin, the hyperboloid `dist_0` default and the $\log_0(\exp_0(v))$ round trip track r down to $r = 10^{-3}$ with a float32 relative error between 0 and 2.4×10^{-7}; the smoothened distance variant (`VERSION_SMOOTHENED`), which floors the spatial radius in quadrature so that the distance is never exactly zero and its gradient stays finite through coincidence, has a relative error of 2.7×10^{-6} at $r = 10^{-3}$ that decays with r (1.6×10^{-6}, 1.1×10^{-6}, 4.3×10^{-7} at the next three grid points), a peak rather than a floor.

H.5 Speed on an NVIDIA H100

Tables 22 and 23 give the compiled and the eager columns of the CUDA benchmark; Figure 5 plots the compiled columns at batch 10^7. The five launches ran back to back as one SLURM job array (`speed_cuda_130`) on a single NVIDIA H100 (driver 575.57.08) of a shared node that was not reserved exclusively, with Python 3.13.7, JAX 0.11.1, and PyTorch 2.13.0 (CUDA 12.9 wheel), over batch sizes 10^2 to 10^7. The tables print

10^3, 10^5, and 10^7. An H100 is dispatch-bound on these operations up to about 10^5 points: the HypLL compiled column sits at a flat 0.05–0.08 ms from 10^2 to 10^5, the geoopt compiled column between 0.06 and 0.43 ms, and the hyperbolix jit column at a median of about 0.07 ms (per-operation 0.06–0.09 ms at 10^2), nearly flat across batch size, which is why the sweep extends to 10^7 and the main-text figure plots the largest batch.

At batch 10^7, compiled against compiled, the picture splits by manifold. On the Poincaré ball, HypLL's kernels are fastest on distance, $\exp_0$, and $\log_0$ in all five launches (1.6–2.3× ahead of hyperbolix, 3.1–12.6× ahead of geoopt) [1]. The two PyTorch libraries are level on Möbius addition and parallel transport, both roughly 2.5× faster than hyperbolix. On the hyperboloid, hyperbolix is 1.9–2.9× faster than geoopt on $\exp_0$, $\log_0$, and parallel transport. The distance has no stable ordering as geoopt's results are bimodal across launches. The gap in the Poincaré ball operations is a compiler limitation, not an algorithmic one. Each of these operations reduces a row to a scalar (e.g. a norm), then rescales the same row by a function of that scalar. A single GPU kernel could perform both steps in a single pass over the data. XLA instead splits the reduction and the rescale into separate kernels, forcing the full (B, D) input to be written out and read back between them. This explains why hyperbolix loses the most in Möbius addition and parallel transport: they have the most reductions per call, and are 2.55× and 2.45× slower than HypLL (Table 22). HLO-level profiling on an A100 corroborates the fusion explanation: the measured HBM-byte ratios for these two operations are 2.57× and 2.45×, closely tracking the wall-clock gap. In contrast, hyperbolix is competitive for $\log_0$ because it has only one reduction and the smallest overhead (1.56×). The two-layer rows are hyperbolix-only: geoopt ships no layers, and HypLL's layer cannot be compiled.

The eager columns (Table 23) compare per-call overhead, not kernels. The hyperbolix eager cell is nearly flat from 10^3 to 10^5 (Möbius addition: 19.8 and 20.4 ms) and rises only at 10^7 (33.6 ms), because re-tracing `vmap` on every call dominates until the kernel itself takes longer than the trace. The PyTorch eager columns rise with batch as expected. A speed-up quoted against the hyperbolix eager column is therefore a Python-overhead ratio; the compiled table is the kernel comparison.

[1] For `dist` and `logmap0`, a data-dependent branch in geoopt's `artan_k` prevents `torch.compile` from tracing the graph, so the compiled column silently falls back to eager mode. The 12.6× and 3.1× ratios for these two operations therefore compare HypLL's compiled kernel against geoopt's host dispatch.

Table 22: Compiled wall-clock time per call (ms) on one NVIDIA H100 for matched primitives and layers, float32, $D = 32$, $c = 1$: geoopt and HypLL under `torch.compile`, hyperbolix under `jax.jit(jax.vmap(·))`. Cell = median of five launch medians, each over 20 repeats; bracket = min–max across the five launches, which share one seed and so bound launch-to-launch state, not sampling. The batches 10^2, 10^4, and 10^6 are in the JSON artifact. n/a cells are structural (HypLL has no hyperboloid, geoopt no layers) except the HypLL Poincaré linear cell, where `torch.compile` fails on HypLL's `ManifoldParameter`. The hyperboloid `dist` geoopt cell is bimodal across launches (4.37–4.40 ms in four, 10.3 ms in one). Script: `scripts/speed_benchmark.py -device cuda`.

Operation	Batch	geoopt (compile)	HypLL (compile)	hyperbolix (jit)
Poincaré `dist`	10^3	0.282 [0.276, 0.393]	0.062 [0.059, 0.065]	0.079 [0.063, 0.098]
	10^5	0.387 [0.337, 0.561]	0.068 [0.062, 0.106]	0.081 [0.066, 0.101]
	10^7	15.4 [15.4, 15.5]	1.22 [1.22, 1.24]	2.76 [2.71, 2.77]
Poincaré $\exp_0$	10^3	0.244 [0.233, 0.254]	0.061 [0.057, 0.061]	0.063 [0.060, 0.069]
	10^5	0.280 [0.268, 0.298]	0.060 [0.059, 0.065]	0.070 [0.065, 0.072]
	10^7	5.13 [5.11, 5.14]	1.26 [1.25, 1.26]	2.66 [2.65, 2.69]
Poincaré $\log_0$	10^3	0.174 [0.170, 0.188]	0.056 [0.054, 0.060]	0.071 [0.070, 0.130]
	10^5	0.213 [0.195, 0.222]	0.058 [0.053, 0.059]	0.068 [0.067, 0.073]
	10^7	3.88 [3.88, 3.90]	1.25 [1.24, 1.26]	1.95 [1.91, 1.99]
Poincaré Möbius $\oplus$	10^3	0.063 [0.061, 0.067]	0.061 [0.060, 0.063]	0.071 [0.062, 0.075]
	10^5	0.070 [0.067, 0.159]	0.065 [0.060, 0.144]	0.102 [0.094, 0.121]
	10^7	1.82 [1.82, 1.82]	1.82 [1.82, 1.82]	4.64 [4.60, 4.64]
Poincaré `ptransp`	10^3	0.061 [0.059, 0.065]	0.064 [0.063, 0.068]	0.066 [0.064, 0.081]
	10^5	0.074 [0.069, 0.114]	0.072 [0.069, 0.119]	0.121 [0.115, 0.151]
	10^7	2.39 [2.37, 2.39]	2.39 [2.37, 2.39]	5.83 [5.82, 5.84]
Poincaré linear (fwd+bwd)	10^3	n/a	n/a	0.141 [0.135, 0.171]
	10^5	n/a	n/a	0.293 [0.281, 0.323]
	10^7	n/a	n/a	19.1 [19.1, 19.2]
Hyperboloid `dist`	10^3	0.079 [0.076, 0.082]	n/a	0.062 [0.060, 0.064]
	10^5	0.096 [0.093, 0.158]	n/a	0.113 [0.110, 0.115]
	10^7	4.39 [4.37, 10.3]	n/a	6.68 [6.67, 6.70]
Hyperboloid $\exp_0$	10^3	0.068 [0.066, 0.071]	n/a	0.076 [0.061, 0.123]
	10^5	0.127 [0.125, 0.132]	n/a	0.110 [0.108, 0.110]
	10^7	7.08 [7.06, 7.41]	n/a	3.65 [3.63, 3.69]
Hyperboloid $\log_0$	10^3	0.077 [0.074, 0.079]	n/a	0.067 [0.060, 0.074]
	10^5	0.137 [0.136, 0.146]	n/a	0.067 [0.066, 0.093]
	10^7	8.20 [8.18, 8.20]	n/a	2.78 [2.73, 2.79]
Hyperboloid `ptransp`	10^3	0.112 [0.106, 0.114]	n/a	0.090 [0.064, 0.101]
	10^5	0.415 [0.410, 0.436]	n/a	0.197 [0.194, 0.234]
	10^7	33.2 [33.2, 33.2]	n/a	16.0 [16.0, 16.1]
Hyperboloid linear (fwd+bwd)	10^3	n/a	n/a	0.080 [0.064, 0.102]
	10^5	n/a	n/a	0.140 [0.138, 0.145]
	10^7	n/a	n/a	6.26 [6.24, 6.27]

Table 23: Eager wall-clock time per call (ms) on the same H100 runs as Table 22: geoopt and HypLL without `torch.compile`, hyperbolix as `jax.vmap` over the single-point API without `jax.jit`. The hyperbolix column measures host-side re-tracing on every call, which is nearly independent of batch size until the kernel dominates; it is not a kernel comparison. Aggregation and n/a semantics as in Table 22.

Operation	Batch	geoopt (eager)	HypLL (eager)	hyperbolix (eager)
Poincaré `dist`	10^3	0.210 [0.197, 0.275]	0.301 [0.280, 0.418]	1.17 [1.13, 1.63]
	10^5	0.245 [0.231, 0.264]	0.313 [0.304, 0.589]	1.60 [1.56, 1.67]
	10^7	15.4 [15.4, 15.4]	20.3 [20.3, 20.4]	5.34 [5.13, 5.88]
Poincaré $\exp_0$	10^3	0.181 [0.177, 0.322]	0.195 [0.188, 0.335]	5.59 [5.13, 6.91]
	10^5	0.216 [0.211, 0.227]	0.212 [0.206, 0.220]	5.84 [5.28, 6.04]
	10^7	9.97 [9.96, 9.98]	11.3 [11.3, 11.3]	7.95 [7.03, 9.98]
Poincaré $\log_0$	10^3	0.134 [0.106, 0.163]	0.078 [0.061, 0.093]	2.69 [2.50, 3.54]
	10^5	0.127 [0.121, 0.132]	0.077 [0.076, 0.077]	2.62 [2.60, 2.74]
	10^7	3.86 [3.84, 3.87]	5.00 [5.00, 5.01]	4.49 [3.77, 6.64]
Poincaré Möbius $\oplus$	10^3	0.203 [0.179, 0.317]	0.228 [0.208, 0.362]	19.8 [19.5, 20.0]
	10^5	0.229 [0.221, 0.237]	0.238 [0.233, 0.533]	20.4 [20.0, 20.7]
	10^7	19.0 [19.0, 19.0]	16.8 [16.8, 16.8]	33.6 [33.5, 33.7]
Poincaré `ptransp`	10^3	0.208 [0.198, 0.291]	0.487 [0.468, 0.688]	23.8 [23.7, 24.5]
	10^5	0.312 [0.307, 0.627]	0.512 [0.499, 0.999]	25.1 [25.0, 25.7]
	10^7	24.7 [24.7, 24.7]	38.6 [38.6, 38.6]	45.3 [45.3, 45.5]
Poincaré linear (fwd+bwd)	10^3	n/a	3.26 [2.13, 3.85]	39.1 [39.0, 40.4]
	10^5	n/a	3.19 [1.55, 4.86]	40.2 [38.9, 40.7]
	10^7	n/a	95.8 [95.7, 95.8]	116 [114, 120]
Hyperboloid `dist`	10^3	0.183 [0.176, 0.190]	n/a	1.62 [1.57, 1.83]
	10^5	0.201 [0.194, 0.212]	n/a	1.53 [1.53, 1.58]
	10^7	18.3 [18.3, 18.3]	n/a	10.5 [10.2, 10.6]
Hyperboloid $\exp_0$	10^3	0.262 [0.238, 0.271]	n/a	12.4 [9.29, 15.2]
	10^5	0.188 [0.183, 0.194]	n/a	13.9 [10.3, 14.9]
	10^7	18.4 [18.4, 18.5]	n/a	17.9 [17.2, 18.6]
Hyperboloid $\log_0$	10^3	0.214 [0.195, 0.308]	n/a	4.39 [3.74, 6.32]
	10^5	0.250 [0.241, 0.278]	n/a	5.95 [5.23, 6.29]
	10^7	19.6 [19.5, 19.6]	n/a	8.28 [8.22, 9.72]
Hyperboloid `ptransp`	10^3	0.421 [0.411, 0.449]	n/a	42.1 [39.6, 44.2]
	10^5	0.624 [0.613, 0.784]	n/a	45.3 [41.8, 46.4]
	10^7	61.4 [61.4, 61.5]	n/a	79.2 [79.1, 81.9]
Hyperboloid linear (fwd+bwd)	10^3	n/a	n/a	12.6 [12.2, 12.7]
	10^5	n/a	n/a	12.6 [12.5, 13.0]
	10^7	n/a	n/a	31.9 [31.1, 32.8]

H.6 Speed on a CPU

The CPU benchmark ran on a two-socket Intel Xeon Silver 4214R (12 cores and 24 hardware threads per socket, 48 logical CPUs) with Python 3.13.8, JAX 0.11.0, and PyTorch 2.13.0 executing on the CPU, over batch sizes 10^2 to 10^5; the tables show 10^5. The machine was shared, though more lightly than in some other launches of this benchmark: the one-minute load average immediately before the first of the five default-thread launches was 0.84.

A CPU comparison between these libraries is a comparison between two runtimes' threading defaults. On this host PyTorch's ATen and Inductor kernels use 24 threads (`torch.get_num_threads()`), and in a three-launch probe at batch 10^5 outside the released tables, the torch columns kept about 24 cores busy (CPU time over wall time) while XLA:CPU kept about 7–8; no `XLA_FLAGS` setting resizes XLA's pool in jaxlib 0.11, and `OMP_NUM_THREADS` moves only PyTorch. We therefore report two tables. Table 25 is each runtime at its default; Table 24 and Figure 6 are a *one-core control*: five further launches of the unmodified script under `taskset -c 2` with `OMP_NUM_THREADS=MKL_NUM_THREADS=1`, which pins all three libraries to one physical core (both the core and its hyperthread sibling were 99–100% idle in the three seconds sampled just before

the first launch, one-minute load average 3.38 at that moment) and holds PyTorch to one thread. `taskset` is the only knob that moves both runtimes, and each JSON records the affinity mask and thread count. The control is the like-for-like comparison; the default table is what a user sees without tuning.

At one core, batch 10^5, compiled against compiled, hyperbolix jit is fastest on four of the nine primitive rows. On three of them the five-launch brackets are disjoint: Poincaré $\log_0$ (1.9× over geoopt, 2.2× over HypLL), hyperboloid $\exp_0$ (1.5× over geoopt), and hyperboloid $\log_0$ (2.2×). On the fourth, Poincaré $\exp_0$, it is 1.6× faster than geoopt but only 1.06× faster than HypLL, with overlapping brackets, so that cell is a tie with HypLL. It loses the other five: HypLL leads on Poincaré distance (2.6×) and Möbius addition (4.6×; geoopt 3.8×); geoopt and HypLL are within noise of each other on Poincaré parallel transport (8.64 versus 8.58 ms) and both are 3.0× faster than hyperbolix there; and geoopt's compiled kernels lead on hyperboloid distance (1.8×) and hyperboloid parallel transport (1.4×), where HypLL has no cells. The default-table gaps are larger (up to ∼12× on Poincaré Möbius addition) because PyTorch uses 24 threads against XLA's ∼8. The one-core control shrinks them to the ratios above, so most, though not all, of the default-table gap is thread-count asymmetry.

The default table carries its own caveats. Its per-column across-launch max/min at 10^5 is 3.89, 3.74, 3.33, 1.26, 1.52, and 1.59 for geoopt eager, geoopt compiled, HypLL eager, HypLL compiled, hyperbolix eager, and hyperbolix jit; the widest hyperbolix jit cell is Poincaré Möbius addition, one launch at 13.2 ms against four at 8.3–9.3 ms, rather than systematic. We did not run a same-day control launch for this table, so we do not quote any single cell to a precision tighter than the reported five-launch bracket. Finally, the CPU and CUDA runs differ in machine, kernel, Python, JAX, and PyTorch build, and the CPU sweep stops at 10^5, so CPU against CUDA is not a single-variable comparison.

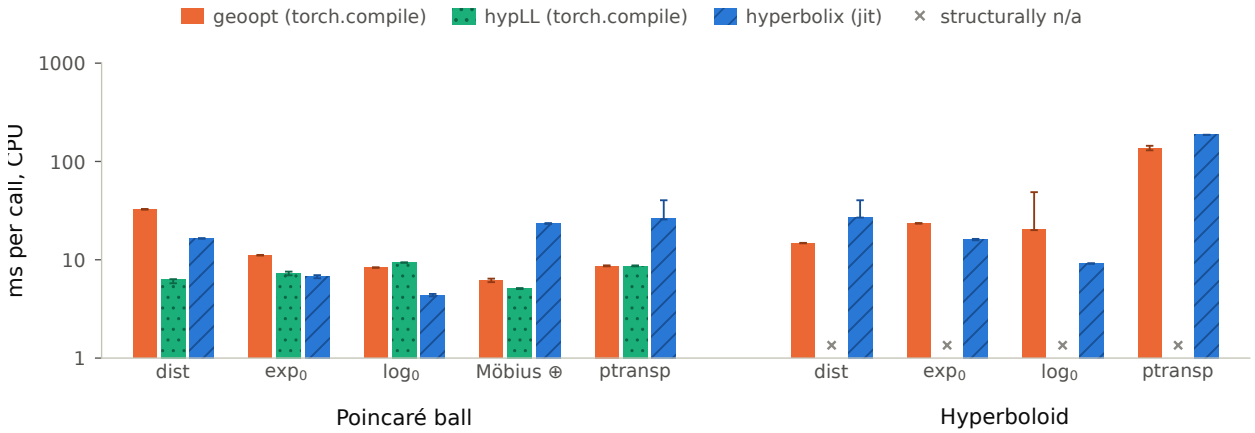

Figure 6: Compiled wall-clock time per call on one CPU core per library (Table 24), batch 10^5, $D = 32$, float32, primitives only; the layer rows and the eager columns are table-only. Bars are the median of five launch medians, error bars the min–max across launches. The axis is logarithmic with a floor at 1 ms, so bar tops, not bar lengths, are proportional to time. The four hyperboloid HypLL slots are structurally absent and drawn as grey crosses.

Table 24: One-core control: wall-clock time per call (ms) at batch 10^5, float32, $D = 32$, with all three libraries pinned to one physical core of the Xeon Silver 4214R (`taskset -c 2`) and PyTorch held to one thread (`OMP_NUM_THREADS=MKL_NUM_THREADS=1`). Cell = median of five launch medians; bracket = min–max across launches. The widest brackets are geoopt's hyperboloid $\log_0$ (max/min 2.4× compiled, 1.6× eager: two launches at ∼48 ms against three at ∼20 ms) and three cells where a single launch departs from the other four: hyperbolix jit on Poincaré parallel transport (1.6×) and hyperboloid distance (1.5×), and geoopt eager on Poincaré parallel transport (1.5×). Hyperboloid parallel transport is among the tightest cells (hyperbolix jit 1.006×). Cause unmeasured. n/a semantics as in Table 22.

Operation	geoopt (eager)	geoopt (compile)	HypLL (eager)	HypLL (compile)	hyperbolix (eager)	hyperbolix (jit)
Poincaré `dist`	32.2 [32.0, 32.6]	32.6 [32.3, 32.9]	29.6 [29.6, 29.9]	6.26 [5.77, 6.35]	39.6 [39.4, 39.7]	16.4 [16.4, 16.6]
Poincaré $\exp_0$	20.5 [20.5, 20.8]	11.1 [11.0, 11.2]	49.7 [49.5, 50.0]	7.23 [6.95, 7.58]	12.3 [12.1, 12.5]	6.82 [6.49, 6.97]
Poincaré $\log_0$	8.06 [8.00, 8.13]	8.33 [8.27, 8.39]	7.54 [7.36, 7.54]	9.35 [9.31, 9.42]	9.96 [9.87, 10.00]	4.29 [4.25, 4.49]
Poincaré Möbius $\oplus$	63.5 [63.3, 64.0]	6.13 [5.92, 6.43]	50.9 [50.8, 51.2]	5.06 [5.03, 5.15]	112 [93.0, 130]	23.4 [23.2, 23.6]
Poincaré `ptransp`	80.9 [54.1, 81.2]	8.64 [8.57, 8.77]	86.8 [63.4, 86.8]	8.58 [8.56, 8.76]	136 [130, 137]	26.1 [25.7, 40.3]
Poincaré linear (fwd+bwd)	n/a	n/a	413 [316, 421]	n/a	336 [313, 348]	181 [181, 181]
Hyperboloid `dist`	25.2 [25.0, 25.7]	14.7 [14.7, 14.9]	n/a	n/a	29.0 [28.8, 42.8]	26.9 [26.9, 40.3]
Hyperboloid $\exp_0$	30.6 [26.3, 30.8]	23.4 [23.3, 23.6]	n/a	n/a	42.2 [41.3, 47.8]	15.9 [15.8, 16.3]
Hyperboloid $\log_0$	34.9 [33.8, 53.3]	20.5 [20.0, 48.7]	n/a	n/a	17.3 [17.2, 17.8]	9.20 [9.10, 9.23]
Hyperboloid `ptransp`	149 [145, 159]	136 [130, 144]	n/a	n/a	261 [234, 266]	186 [186, 187]
Hyperboloid linear (fwd+bwd)	n/a	n/a	n/a	n/a	77.0 [74.9, 78.1]	25.2 [24.9, 25.3]

Table 25: Runtime-default threading: wall-clock time per call (ms) at batch 10^5, float32, $D = 32$, on the same Xeon Silver 4214R with PyTorch at its default 24 threads and XLA:CPU at its default pool (one-minute load 0.84 at launch). Cell = median of five launch medians; bracket = min–max across launches; the brackets mix co-tenancy with launch-to-launch state. n/a semantics as in Table 22.

Operation	geoopt (eager)	geoopt (compile)	HypLL (eager)	HypLL (compile)	hyperbolix (eager)	hyperbolix (jit)
Poincaré `dist`	5.62 [5.30, 6.00]	5.72 [5.69, 6.19]	5.69 [5.14, 5.80]	0.949 [0.879, 1.10]	24.5 [24.0, 25.6]	10.1 [9.87, 10.8]
Poincaré $\exp_0$	3.56 [3.35, 13.0]	2.22 [1.85, 2.77]	13.1 [12.4, 13.4]	1.22 [1.21, 1.40]	6.05 [5.96, 6.45]	2.49 [2.44, 2.63]
Poincaré $\log_0$	1.65 [1.55, 1.74]	1.80 [1.71, 1.93]	1.10 [1.00, 1.23]	0.735 [0.669, 0.780]	4.02 [3.89, 4.34]	1.91 [1.87, 1.92]
Poincaré Möbius $\oplus$	14.2 [13.9, 14.3]	0.844 [0.758, 0.853]	4.30 [3.85, 12.8]	0.719 [0.620, 0.772]	46.1 [37.8, 57.4]	8.86 [8.33, 13.2]
Poincaré `ptransp`	6.99 [6.09, 7.80]	1.08 [1.00, 1.10]	8.15 [7.98, 8.63]	1.10 [1.03, 1.13]	53.5 [50.7, 53.8]	7.09 [7.01, 7.25]
Poincaré linear (fwd+bwd)	n/a	n/a	86.1 [75.9, 89.2]	n/a	143 [128, 150]	80.8 [80.1, 81.2]
Hyperboloid `dist`	4.59 [4.34, 4.85]	1.93 [1.49, 2.01]	n/a	n/a	8.85 [8.69, 9.10]	6.12 [6.04, 6.14]
Hyperboloid $\exp_0$	5.66 [4.98, 6.68]	7.63 [7.50, 8.29]	n/a	n/a	19.7 [18.5, 20.3]	9.88 [9.53, 10.1]
Hyperboloid $\log_0$	18.5 [6.90, 19.4]	30.4 [8.66, 32.4]	n/a	n/a	7.81 [7.51, 8.39]	3.27 [3.21, 3.29]
Hyperboloid `ptransp`	41.7 [39.6, 46.0]	88.6 [86.0, 91.9]	n/a	n/a	111 [100, 132]	61.4 [58.5, 61.9]
Hyperboloid linear (fwd+bwd)	n/a	n/a	n/a	n/a	31.4 [29.8, 39.9]	6.78 [6.64, 6.84]

H.7 Two-point cancellation on the hyperboloid

Every accuracy number so far comes from an operation anchored at the origin: $\exp_0$, $\log_0$, dist_0, and ptransp from the origin. The hyperboloid's two-point Minkowski inner product $\langle x, y\rangle_{\mathcal{L}} = -x_0 y_0 + \langle x_s, y_s\rangle$ is different: away from the origin, both terms grow like e^a for a point at geodesic radius a (with $a = \sqrt{c}\, d(o, x)$), so their difference is a subtraction of two $O(e^{2a})$ quantities, a cancellation no origin-anchored operation exercises. We test it directly, with a construction that needs no oracle: in geodesic polar coordinates $\mathrm{polar}(r, u) = (\cosh(\sqrt{c}\, r)/\sqrt{c},\ \sinh(\sqrt{c}\, r)/\sqrt{c} \cdot u)$ for a unit direction u, two points on the same ray at radii a and $a+1$ are exactly geodesic distance 1 apart by construction, and the tangent vector $v = \big((\sinh a,\ \cosh a\, u) + (0, w)\big)/\sqrt{2}$ at the point at radius a, with w a unit vector orthogonal to u. It has Riemannian norm exactly 1 because it is the normalized sum of the ray's unit-speed velocity and a unit tangent orthogonal to it, and it points at 45° to the ray. Ground truth is therefore a constant, not a transcribed formula, so no oracle-transcription error can enter; only the library's own arithmetic can produce a deviation. We sweep a from 2 to 16 at $c = 1$, dimension 8, 500 sampled directions per cell, both dtypes, comparing hyperbolix against geoopt's `Lorentz` model. We evaluate the tangent norms for both libraries on the same set of vectors.

Table 26 reports the four radii cited by the accuracy claims below; the corresponding main-text figure (`results_paper/hyperboloid_cancellation.pdf`) plots the full sweep. On the distance, both libraries' error grows like e^{2a}, consistent with the cancellation model, which makes this a slope claim rather than a

pass/fail at one radius. They differ in the prefactor: geoopt computes the difference literally, and its float32 distance median passes 1% error at $a \approx 7$ and returns `NaN` from $a = 9$ onward (its `arcosh` receives a negative argument once the cancellation consumes every significant digit). hyperbolix's cancellation-free form sits at the float32 resolution floor out to $a \approx 10$: through $a = 12$ its float32 maximum absolute deviation is 3.7×10^{-4} (median 5.6×10^{-5}), and its float64 deviation is $\leq 6.7 \times 10^{-16}$ (3 ulp) at every radius we tested, including $a = 16$. The model prediction is exact enough to check quantitatively: geoopt's float64 curve is its float32 curve shifted by $\Delta a = 10$, matching $\varepsilon_{64}/\varepsilon_{32} = 2^{-29}$ (median 3.43×10^{-3} at $a = 16$ in float64, the same 3.43×10^{-3} at $a = 6$ in float32).

hyperbolix is not unconditionally safe in float32; it leaves the resolution floor about five units of radius later than geoopt. Its own float32 median distance error reaches 7.2×10^{-4} at $a = 14$ and 1.0×10^{-1} at $a = 16$, both outside the table's printed range but visible in the main-text figure, which is why the sweep runs to $a = 16$ rather than stopping once geoopt has failed. Two hedges apply when quoting this comparison: above $a = 8$, geoopt's maximum (as opposed to its median) is reduction-order noise rather than a stable quantity, since every digit in a fully-cancelled float32 cell is meaningless – quote its median, or simply "$O(1)$/`NaN`"; and the timed callables (Table 26 omits timing) show cost flat with radius for hyperbolix jit (0.95–1.72 ms) and for geoopt eager (0.25–0.81 ms) across $a = 2$ to 16, a single launch with unequal thread pools between the two libraries, so it answers only "does cost grow with radius" and is not a speed claim in either direction.

Table 26: Two-point Minkowski-inner-product cancellation on the hyperboloid, $c = 1$, dimension 8, 500 sampled directions per cell: relative max|dev| (median|dev|) against the exact constructions of Section H.7. Both true values are 1, so relative and absolute deviation coincide. hyperbolix 1.3.0 against geoopt 0.5.1's `Lorentz` model. geoopt's distance is fully cancelled from $a = 9$ (`NaN`); its maxima at $a \geq 8$ are reduction-order noise and only its medians are meaningful. hyperbolix's distance stays at float32 rounding level through $a = 10$ and within 6.7×10^{-16} in float64 at every radius; its tangent norm stays within 1.35× of the input-rounding error from $a = 6$. Script: `scripts/hyperboloid_cancellation.py`, CPU backend.

		dist		tangent norm	
a	dtype	hyperbolix	geoopt	hyperbolix	geoopt
6	float32	2.38×10^{-7} (5.96×10^{-8})	1.69×10^{-2} (3.43×10^{-3})	6.56×10^{-6} (7.15×10^{-7})	2.93×10^{-3} (9.76×10^{-4})
6	float64	6.66×10^{-16} (0)	6.24×10^{-11} (1.29×10^{-11})	1.67×10^{-14} (1.78×10^{-15})	7.28×10^{-12} (1.82×10^{-12})
8	float32	3.58×10^{-7} (0)	1.00×10^{0} (3.76×10^{-2})	4.73×10^{-5} (4.98×10^{-6})	1.34×10^{-1} (6.46×10^{-2})
8	float64	4.44×10^{-16} (2.22×10^{-16})	2.70×10^{-9} (4.71×10^{-10})	1.13×10^{-13} (1.20×10^{-14})	3.49×10^{-10} (1.16×10^{-10})
10	float32	2.50×10^{-6} (2.38×10^{-7})	NaN (NaN)	4.07×10^{-4} (3.81×10^{-5})	1.83×10^{0} (1.00×10^{0})
10	float64	2.22×10^{-16} (0)	1.58×10^{-7} (4.48×10^{-8})	8.80×10^{-13} (8.15×10^{-14})	1.86×10^{-8} (3.73×10^{-9})
12	float32	2.89×10^{-4} (4.53×10^{-5})	NaN (NaN)	2.52×10^{-3} (2.64×10^{-4})	2.67×10^{1} (1.50×10^{1})
12	float64	4.44×10^{-16} (2.22×10^{-16})	7.56×10^{-6} (1.07×10^{-6})	7.37×10^{-12} (7.15×10^{-13})	7.15×10^{-7} (2.38×10^{-7})